INTELLIGENT AYURVEDIC HERBAL PLANT IDENTIFICATION AND ECONOMIC ANALYSIS IN SRI LANKA

INDIVIDUAL THESIS

Pattiya Arachchi P A H K

(IT22112132)

Individual component: **Explainable AI System for Ayurvedic Medicinal Plant Quality Assessment**

Group ID: **R26-IT-021**

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Pattiya Arachchi P A H K

Student ID: IT22112132

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Nelum Amarasena		
Co-supervisor: Ms. Kaushika Kahatapitiya		

ABSTRACT

Traditional ayurvedic medicine is based on the strict quality control of the medicinal leaves of plants and raw herbal resources, with the effectiveness of treatment directly depending on the ability to select healthy and mature parts of the plant. But the traditional quality evaluation is based on manual, visual assessment, which is subjective, labor-intensive and easy to be human mistake. While deep learning models have proven to be highly accurate in plant classification, their opaque and "black-box" decision-making process can hinder trust and adoption in critical sectors such as medical and agricultural supply chains.

To overcome these problems, the present work proposes a decision support system based on Explainable Artificial Intelligence (XAI) approach for Ayurvedic medicinal plant leaf evaluation. The main method is a hierarchical, three-layer deep learning framework with a transfer learning approach, which is based on the EfficientNetV2-B0 model. Stage 1 is an implementation of the pipeline that classifies the plant species with confidence thresholds and an Unknown class that filters out un-supported input. In Stage 2, an embedded shared convolutional neural network (CNN) is used to classify the visible leaf health conditions (SHC) (healthy or diseased) using species-specific output heads. For supported species, developmental maturity (e.g., young or mature leaves) is assessed in Stage 3 (developmental maturity).
Model classifications are connected to domain knowledge bases providing detailed descriptions and medicinal suitability categories, to ensure actionable transparency. Moreover, Gradient-weighted Class Activation Mapping (Grad-CAM) can produce visual heatmaps that represent the specific anatomical areas of the leaves (e.g. necrotic lesions or maturity patterns) that contribute to each prediction.

Empirical testing shows high classification accuracy across species, condition and maturity stages, and reliable visual explanations for the model reasoning which enable non-expert users to verify model reasoning. The main technical achievement is the design, implementation and integration of this species-aware quality assessment pipeline that is staged and used in a web and mobile framework. This system offers an open screening tool which helps to improve the quality control, traceability and trust among stakeholders in the traditional herbal supply chain in Sri Lanka.

Keywords: Ayurvedic Medicinal Plants, Deep Learning, Quality Assessment, Explainable AI (XAI), Grad-CAM

ACKNOWLEDGEMENTS

We sincerely thank our supervisor, Mr. Nelum Amarasena, and our co-supervisor, Ms. Kaushika Kahatapitiya, for their guidance, constructive criticism and steady support throughout this research. Their feedback on novelty and measurability shaped the evaluation design of the whole project.

We are grateful to the panel members who reviewed the project at each stage, and in particular for the request that led to the flexible audience design of the knowledge assistant. We thank the Department of Information Technology of the Sri Lanka Institute of Information Technology for the facilities and resources provided, and the students and members of the public who tried the prototype and shared their comments.

We also acknowledge the researchers, dataset contributors and open-source communities whose work supported the technical foundation of the system, and our families and friends for their encouragement during the final year of our degree.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 1

1.3 Problem statement 2

1.4 Research gap 2

1.5 Aim and objectives 3

1.5.1 Main Aim 3

1.5.2Specific Objective 3

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 4

1.7.1 Scope 4

1.7.2 Limitations 4

1.7.3 Assumption 5

1.8 Individual contribution 5

1.9 Thesis structure 5

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review method and scope 7

2.2 Domain and theoretical foundations 7

2.2.1 Botanical Quality Assessment in Traditional Medicine 7

2.2.2 Computer Vision and Deep Learning in Agriculture 8

2.2.3 Transfer Learning Backbones 8

2.2.4 Explainable AI (XAI) and Grad-CAM Principles 9

2.3 Related research 10

2.3.1 Automated Species Identification of Medicinal Plants 10

2.3.2 Plant Disease and Visual Quality Detection 10
2.3.3 Explainable AI Applications in Agricultural Diagnostics 10
2.4 Existing systems products and techniques 11
2.4.1 Global General-Purpose Identification Tools 11
2.4.2 Local and Domain-Specific Herbal Applications 11
2.4.3 Handcrafted Feature Extraction vs. End-to-End Deep Learning 11
2.5 Gap synthesis and conceptual framework 12
2.5.1 Summary Matrix of Related Works 12
2.5.2 Synthesized Research Gaps 13
2.5.3 Conceptual Framework of the Proposed System 14
3 RESEARCH METHODOLOGY 16
3.1 Research design 16
3.2 Data participants or study materials 17
3.2.1 Dataset Composition and Stage Distribution 17
3.2.2 Image Preprocessing & Aspect-Preserving Letterboxing 19
3.2.3 Data Augmentation Strategies 19
3.3 Experimental or study procedure 20
3.3.1 Model Architectures and Transfer Learning Configurations 20
3.3.2 Model Training Hyperparameters 20
3.3.3 Pre-Inference Validation and Threshold Routing 21
3.3.4 Multi-Image Probability Aggregation 21
3.3.5 Grad-CAM Visual Explainability Setup 21
3.4 Measures and evaluation metrics 22
3.5 Data analysis 22
3.6 Ethics privacy security and responsible AI 23
3.7 Reproducibility and research artefacts 23

4 PRODUCT AND SOLUTION DESIGN 25
4.1 Requirements 25
4.1.1 Functional Requirements 25
4.1.2 Non-Functional Requirements 26
4.2 Component architecture within the complete solution 27
4.2.1 Overall Platform Integration Context 27
4.2.2 High-Level Three-Tier Component Architecture 29
4.2.3 End-to-End Execution Flow 30
4.3 Design alternatives and decisions 30
4.3.2 Backbone Model Selection: EfficientNetV2-B0 vs. MobileNetV3-Large vs. ResNet50 32
4.3.3 Multi-Head Shared Backbone vs. Independent Model Files 32
4.3.4 Explainability Approach: Grad-CAM vs. LIME vs. SHAP 32
4.4 Detailed design 33
4.4.1 Data Preprocessing & Input Standardization Pipeline 33
4.4.2 Multi-Image Probability Aggregation Algorithm 34
4.4.3 REST API Contracts & Schema Definitions 34
4.4.4 Domain Knowledge Base Schema 36
4.4.5 Grad-CAM Heatmap Generation Algorithm 37
4.5 Standards and professional practice 38
4.5.1 Software Engineering & RESTful API Standards 38
4.5.2 Responsible AI, Safety, & Ethics Standards 38
4.5.3 Code Quality & Repository Standards 39
5 IMPLEMENTATION INTEGRATION AND TESTING 40
5.1 Development environment and technology stack 40
5.2 Component implementation 41

5.2.1 Image Validation Module (quality.image_validator.py) 41
5.2.2 Stage 1 Species Identification & Out-of-Distribution Rejection (quality.plant_identifier.py) 43
5.2.3 Stage 2 Species-Conditioned Health Classifier (quality.condition_classifier.py) 46
5.2.4 Stage 3 Conditional Maturity & Decision Fusion (quality.maturity_classifier.py & quality.maturity_decision.py) 51
5.2.5 Grad-CAM Visual Explainability Processor (quality.gradcam.py) 53
5.3 Integration with the other components 56
5.4 Verification and software testing 58
5.4.1 Executed Automated Unit Test Suite 58
5.4.2 Synthetic Validation & Model Smoke Probes 59
5.4.3 Isolated Stage Model Performance Metrics 60
5.5 Deployment and operation 61
5.5.1 Microservice Configuration & Model Lifecycle 61
5.5.2 Client Timeout & Fallback Operational Rules 61
6 RESULTS AND VALIDATION 63
6.1 Research results 63
6.1.1 Stage 1: Species Identification & Out-of-Distribution Rejection Results 63
6.1.2 Stage 2: Species-Conditioned Health Condition Results 64
6.1.3 Stage 3: Conditional Maturity Classification Results 65
6.1.4 Deep Learning Backbone Model Benchmarking 66
6.2 Product and technical results 66
6.2.1 Operational Execution Latency 67
6.2.2 Automated Image Validation Performance 67
6.2.3 Grad-CAM Heatmap Technical Verification 68

6.3 User or stakeholder validation 68
6.3.1 Manual Evidence Fusion & Decision Support 68
6.3.2 Automated Software Unit Testing Suite 69
6.3.3 Ethical & Advisory Boundary Validation 69
6.4 Integration results 70
6.4.1 Multi-Image Probability Aggregation 70
6.4.2 Cross-Pillar Interoperability & Platform Workflow 70
6.5 Summary of findings 72
7 DISCUSSION 73
7.1 Interpretation of findings 73
7.1.1 Stage 1: Species Identification and Out-of-Distribution Rejection 73
7.1.2 Stage 2: Species-Conditioned Health Condition Assessment 74
7.1.3 Stage 3: Healthy-Branch Conditional Maturity Classification 74
7.1.4 Multi-Image Probability Aggregation Impact 75
7.2 Comparison with literature baselines and existing solutions 75
7.2.1 Comparison with Global and Local Systems 76
7.2.2 Deep Learning Backbone Architecture Benchmarking 76
7.3 Research contribution 77
7.4 Product contribution 78
7.5 Limitations and threats to validity 79
7.6 Implications 79
7.6.1 Theoretical and Academic Implications 79
7.6.2 Clinical and Agricultural Practice Implications 80
8 CONCLUSION AND FUTURE DIRECTIONS 81
8.1 Conclusion 81

8.2 Achievement of objectives 82
8.3 Individual research and product contributions 83
8.3.1 Individual Research Contributions 83
8.3.2 Individual Product and Engineering Contributions 84
8.4 Future directions 85
REFERENCES 87
GLOSSARY ..... **Error! Bookmark not defined.**
APPENDICES ..... **Error! Bookmark not defined.**
Appendix A Research instruments or extended results **Error! Bookmark not defined.**
Appendix B Academic integrity and AI use disclosure **Error! Bookmark not defined.**

LIST OF FIGURES

Figure 1: Conceptual Framework of the Proposed Ayurvedic Medicinal Plant Quality Assessment System 14
Figure 2: Research Methodology and Three-Stage Classification Workflow 17
Figure 3: Overall Platform Architecture and Integration of the Quality Assessment Component 28
Figure 4 : Three-Tier Architecture of the Quality Assessment System 29
Figure 5: End-to-End Execution Flow of the Medicinal Plant Quality Assessment Pipeline 30
Figure 6: Comparison of Single-Stage and Hierarchical Multi-Stage Classification Architectures 31
Figure 7: Development Environment and Runtime Technology Stack 40
Figure 8: Species-Conditioned Multi-Head CNN Architecture for Health Classification 47
Figure 9: Integration of the Quality Assessment Microservice with the Conversational Knowledge Assistant 57
Figure 10: Classification Performance Across the Three Deep Learning Pipeline Stages 61
Figure 11: Cross-Component Integration and Context Handoff Workflow 71

LIST OF TABLES

Table 1: Comparative Analysis of Existing Research on Medicinal Plant Identification and Quality Assessment 13
Table 2: Dataset Distribution Across Training, Validation, and Testing Sets for the Three Classification Stages 18
Table 3: Functional Requirements of the Ayurvedic Medicinal Plant Quality Assessment System 26
Table 4: Automated Unit Test Cases and Results for the Maturity Assessment Module 59
Table 5: Synthetic Image Validation and Model Smoke Test Results 60
Table 6: Classification Performance Summary Across the Three Deep Learning Pipeline Stages 63
Table 7: Stage 1 Species Identification Performance by Class 64
Table 8: Stage 2 Health Condition Classification Performance Across Supported Plant Species 64
Table 9: Stage 3 Maturity Classification Performance Across Supported Plant Species 66
Table 10: Comparative Performance of EfficientNetV2-B0, MobileNetV3-Large, and ResNet50 Architectures 66
Table 11: Average Execution Latency of Quality Assessment System Components 67
Table 12: Grad-CAM Heatmap Feature Localization Across Different Leaf Conditions 68
Table 13: Automated Maturity Decision-Support Unit Testing Results 69
Table 14:Summary of Isolated Classification Performance Across the Three Pipeline Stages 73
Table 15: Comparison of the Proposed Quality Assessment System with Existing Global and Sri Lankan Solutions 76
Table 16: Deep Learning Backbone Architecture Benchmark Comparison 77
Table 17: Achievement of Individual Research Objectives and Supporting Validation Evidence 83

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BGR	Blue, Green, Red (Color Space)
CAM	Class Activation Mapping
CCE	Categorical Cross-Entropy
CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
cuDNN	CUDA Deep Neural Network library
DCNN	Deep Convolutional Neural Network
DL	Deep Learning
EEE	EfficientNetV2-B0 (Stage 1) + EfficientNetV2-B0 (Stage 2) + EfficientNetV2-B0 (Stage 3) Pipeline
F1-Score	Harmonic Mean of Precision and Recall
FLOPs	Floating Point Operations
FR	Functional Requirement
GLCM	Gray-Level Co-occurrence Matrix
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
HSV	Hue, Saturation, Value (Color Space)
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
ImageNet	ImageNet Large Scale Visual Recognition Challenge Dataset
IP	Intellectual Property
JET	Joint Environmental Training / JET Colormap Palette
JSON	JavaScript Object Notation
KB	Knowledge Base

LIME	Local Interpretable Model-agnostic Explanations
MBConv	Mobile Inverted Bottleneck Convolution
ML	Machine Learning
MLP	Multi-Layer Perceptron
NAS	Neural Architecture Search
NFR	Non-Functional Requirement
OOD	Out-of-Distribution
OpenCV	Open Source Computer Vision Library
PEP	Python Enhancement Proposal
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RGB	Red, Green, Blue (Color Space)
SaaS	Software as a Service
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
TFLite	TensorFlow Lite
UI	User Interface
UN	United Nations
VGG	Visual Geometry Group
XAI	Explainable Artificial Intelligence
YOLO	You Only Look Once

1 INTRODUCTION

1.1 Overall project context

Sri Lanka's traditional Ayurvedic medicine has been a primary healthcare system in the country for more than 3000 years and provides natural remedies and primary healthcare for a large percentage of the population [1] [2] [3]. Authentic identification and quality of raw medicinal plant materials are crucial for the therapeutic activity and safety of Ayurvedic treatments [4]. But according to classical literature almost 30 per cent of medicinal plants are prone to botanical controversies and visual misidentification because of the morphological similarity among species [5].

To overcome these systemic problems, the research project "Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka" is to develop an integrated Digital Platform. The end-to-end system integrates computer vision, explainable AI (XAI), natural language processing, and agricultural economics to enable farmers, herbal gatherers, Ayurvedists, and consumers [2]. The complete platform encompasses four key functional pillars:

1. Automated medicinal plant species identification.
2. Explainable medicinal plant quality assessment.
3. Conversational knowledge retrieval and treatment guidance.
4. Cultivation advisory and market price forecasting.

1.2 Individual component context

The present dissertation is a specific component of the overarching research project “Explainable AI System for Ayurvedic Medicinal Plant Quality Assessment”.

The main objective of this component is to develop an automated tool for visual screening and decision support that assesses the quality of leaves of medicinal plants used in Ayurveda. To this end, the component is based on a hierarchical three-stage deep learning pipeline which is deployed via a Flask REST backend and Flutter mobile/web interface:

- **Stage 1 (Species Identification):** Identifies whether an uploaded leaf image belongs to one of the supported species or assigns it to an *Unknown* class if confidence falls below defined thresholds (e.g., 60%).
- **Stage 2 (Condition/Health Assessment):** Utilizes a species-conditioned, multi-head Convolutional Neural Network (CNN) to evaluate visible health conditions (such as healthy, leaf spot, leaf rot, or insect damage).
- **Stage 3 (Maturity Classification):** Evaluates developmental maturity (e.g., young vs. mature) specifically for healthy leaves of species that visually support maturity differentiation.

The active system supports six key Ayurvedic species in Sri Lanka: Betel (Piper betle), Centella asiatica (Gotu Kola), Holy Basil (Tulsi), Kalanchoe pinnata (Akkapana), Murraya koenigii (Curry leaf / Karapincha), and Neem (Kohomba). For explainability, the pipeline integrates Gradient-weighted Class Activation

Mapping (Grad-CAM) to overlay visual heatmaps on input images, illustrating the exact leaf regions driving model decisions.

1.3 Problem statement

Quality evaluation is mostly done by manual, visual assessment by experienced taxonomists or Ayurvedic doctors in the traditional herbal supply chains [6]. This manual process has several serious issues:

- **Subjectivity and Human Error:** Manual inspection is prone to inconsistency, cognitive fatigue, and human error, leading to diseased, pest-damaged, or improperly mature leaves entering medicine production [6].
- **Scarcity of Domain Expertise:** Professionally trained taxonomists and experienced Ayurvedic practitioners are scarce, especially in rural harvesting areas, leading non-experts to collect incorrect or compromised plant parts [6] [7].
- **Efficacy and Safety Risks:** Utilizing compromised or improperly aged plant materials reduces therapeutic potency and risks patient health [7].
- **The "Black-Box" AI Trust Barrier:** Standard deep learning models provide classification labels without explaining their reasoning [8]. Without visual evidence showing what visual features the model prioritized, Ayurvedic practitioners and quality assurance managers cannot confidently trust AI predictions in clinical or pharmaceutical applications

1.4 Research gap

Existing literature highlights significant progress in automated plant identification, but critical gaps remain in medicinal plant quality assessment:

- **Lack of Interpretability in Quality Screening:** State-of-the-art vision models (e.g., YOLO, generic CNNs) prioritize top-1 accuracy over explainability, providing no visual justification for quality predictions.
- **Absence of Species-Conditioned Multi-Stage Architecture:** Existing plant disease detectors evaluate general crop diseases without conditioning quality and maturity parameters on species-specific morphological characteristics.
- **Disconnect Between Visual AI and Knowledge Bases:** Automated vision tools rarely link visual classification and XAI heatmaps with structured domain knowledge detailing medicinal suitability, treatment implications, and harvesting recommendations.

1.5 Aim and objectives

1.5.1 Main Aim

Design, implement and evaluate an explainable AI decision support system to identify selected medicinal plant species of Ayurveda, and visible health condition, maturity stage and medicinal suitability of the identified plant species based on leaf images, Grad-CAM visual heatmaps and domain knowledge bases.

1.5.2Specific Objective

1. **Species Identification & Rejection:** Create a transfer learning model that will classify supported Ayurvedic medicinal plant species with confidence level and reject the unsupported inputs as "Unknown class".
2. **Species-Conditioned Health Classification:** Develop a multi-head CNN to diagnose the visible health conditions for each species with a species-specific diagnostic category.
3. **Conditional Maturity Assessment:** Evaluate developmental maturity for healthy leaves of species which can be evaluated for visual maturity.
4. **Architecture Benchmarking:** Run and compare EfficientNetV2-B0 and MobileNetV3-Large architectures based on speed, model size and accuracy.
5. **Confidence-Aware Handling:** Add rejection thresholds to indicate confident and uncertain predictions.
6. **Grad-CAM Explanation Generation:** Create Gradient-weighted Class Activation Mapping (Grad-CAM) heatmaps of anatomical leaf regions that affect condition and maturity predictions.
7. **Knowledge Base Integration:** Connect model classification outputs to structured disease and maturity knowledge bases providing descriptions and medicinal suitability guidance.
8. **Manual Evidence Fusion:** Explore a layer for evidence fusion that combines user-observed manual leaf characteristics (e.g., texture, dimensions) with CNN outputs.
9. **Full-Stack Application Deployment:** Deploy the end-to-end staged pipeline in a Flask REST backend and Flutter web/mobile application.

10 **Boundary & Limitation Identification:** Establish the real, visual and plant level limitation of image based quality assessment.

1.6Research questions or hypotheses

- **RQ1:** How can a hierarchical, multi-stage deep learning pipeline effectively structure species identification, health condition classification, and maturity assessment for medicinal leaves?
- **RQ2:** To what extent does a species-conditioned multi-head CNN architecture improve classification granularity over single-head generic classifiers?
- **RQ3:** How can visual explanations via Grad-CAM be integrated to provide verifiable visual evidence of model reasoning for non-expert users?
- **RQ4:** How effective are multi-image probability aggregation and confidence thresholding in handling real-world operational inputs?

1.7 Scope limitations and assumptions

1.7.1 Scope

- Focuses on six selected Ayurvedic species in Sri Lanka: Betel, Centella asiatica, Holy Basil, Kalanchoe pinnata, Murraya koenigii, and Neem
- Assesses surface health of leaves and visual maturity and growth stages from visible RGB digital images.
- Computes the probability distribution over multiple uploaded up to 3 leaf images.

1.7.2 Limitations

- Visual Screening Only: The system evaluates visible morphological characteristics; it does not measure chemical composition, active alkaloid/phytochemical potency, or biological toxicity.

- Input Quality Dependency: Image validation checks resolution, blur, and brightness, but cannot guarantee that an image actually contains a medicinal leaf.
- Single Leaf Focus: Evaluates individual leaf samples rather than whole-plant or bulk crop health.

1.7.3 Assumption

- Input images depict clear, well-lit views of plant leaves without severe occlusion.
- Knowledge base references accurately reflect standard Ayurvedic medicinal literature.

1.8 Individual contribution

The primary individual technical contribution of this research includes:

1. Hierarchical Staged Architecture: Design and implement the end-to-end multi-stage pipeline that takes species selection and species-head condition classification, and then returns healthy-branch maturity classification.
2. Multi-Head Softmax Classifiers: Training common backbone CNN models for different species and conditions of maturity with separate softmax output heads.
3. Multi-Image Probability Aggregation: Use of a multi-view inference engine which takes the average of the probability distributions obtained from up to three leaf views.
4. Visual Interpretability Integration: Integrating real-time Grad-CAM heatmap generation for condition and maturity predictions into the Flask/Flutter application.
5. Knowledge-Driven Decision Support: Organizing and embedding domain CSV knowledge bases to convert visual predictions to actionable medicinal suitability descriptions and recommendations.

1.9 Thesis structure

The rest of this dissertation is structured in the following manner:

- **Chapter 2:** Literature Review and Related Work — Provides literature review and related work for Computer Vision in Agriculture, Transfer Learning, Explainable AI (Grad-CAM), and a focus on research gaps.
- **Chapter 3:** Research Methodology - provides a description of the experimental research design, dataset collection, dataset preprocessing, training configurations and evaluation metrics.
- **Chapter 4:** Product and Solution Design - Specifies functional requirements, system architecture, data flow, API schemas and interactions between components.
- **Chapter 5:** Implementation, Integration, and Testing — Provides comprehensive implementation and integration information for all the components (Flask, Flutter, TensorFlow), along with software verification.
- **Chapter 6:** Results and Validation — Analyzes the individual stage metrics, confusion matrix, multi-image aggregation performance and Grad-CAM visualization outputs.
- **Chapter 7:** Discussion — Discusses findings, compares against literature baselines, discusses error cases, and discusses threats to validity.
- **Chapter 8:** Commercialisation and Sustainability — Describes the personas of target markets, value propositions, B2B/SaaS business models and connection to UN Sustainable Development Goals.
- **Chapter 9:** Conclusion and Future Directions — Summarize the accomplishment of research objectives, key contributions, and roadmap for future research.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

To establish a rigorous theoretical foundation for an explainable Ayurvedic medicinal plant quality assessment system, a systematic review of peer-reviewed literature was conducted across major academic databases, including IEEE Xplore, Elsevier ScienceDirect, SpringerLink, and BMC Plant Biology [5] [2] [8] [9]. The review scope encompasses research published between 2010 and 2025, prioritizing recent advancements in computer vision, deep learning backbones, explainable artificial intelligence (XAI), and domain-specific botanical platforms [5] [8] [10] [7]. Literature search queries focused on four core axes:

1. Classical botanical misidentification and Ayurvedic quality assessment challenges [5].
2. Convolutional Neural Network (CNN) architectures and transfer learning backbones for plant classification [5] [8] [11].
3. Explainable AI frameworks, particularly Gradient-weighted Class Activation Mapping (Grad-CAM) in agricultural diagnostics [9] [8] [10].
4. Existing commercial and academic mobile applications for medicinal plant identification [2] [7] [11].

2.2 Domain and theoretical foundations

2.2.1 Botanical Quality Assessment in Traditional Medicine

Traditional Ayurvedic medicine relies fundamentally on raw plant materials—predominantly leaves—whose therapeutic efficacy depends directly on accurate species identification, health condition, and stage of maturity [5]. However, classical botanical literature indicates that approximately **30% of medicinal plants mentioned in Ayurvedic texts remain controversial or subject to misidentification** due to close morphological similarities, nomenclatural ambiguities across regional dialects, and the lack of professional taxonomic expertise among

local collectors [5]. Manual visual inspection is inherently subjective, labor-intensive, and susceptible to cognitive fatigue, leading to diseased, pest-damaged, or under-mature leaves entering herbal processing supply chains [5].

2.2.2 Computer Vision and Deep Learning in Agriculture

Early computer vision approaches for leaf identification relied heavily on handcrafted feature extraction, including Gray-Level Co-occurrence Matrices (GLCM) for texture analysis, color histograms, and morphological contour descriptors [2] [11] [12]. While these traditional machine learning models achieved reasonable accuracy under strictly controlled laboratory conditions, they exhibited weak generalization when encountering field variations in lighting, background noise, and leaf surface deformities [5] [8] [11].

The emergence of **Convolutional Neural Networks (CNNs)** revolutionized visual plant diagnostics by automating hierarchical feature extraction directly from raw RGB images [8] [11] [5]. Convolutional layers automatically learn low-level visual primitives (edges, textures) in early layers and high-level semantic structures (venation patterns, lesion spot geometry, leaf margins) in deeper layers, significantly outperforming handcrafted feature pipelines [11] [5] [8].

2.2.3 Transfer Learning Backbones

Training deep CNN architectures from scratch requires massive annotated datasets and substantial computational resources [7]. **Transfer learning** mitigates these constraints by leveraging deep representations pre-trained on large-scale datasets such as ImageNet [8] [7]. Several key backbones dominate recent literature:

- **EfficientNet / EfficientNetV2:** Utilizes compound scaling to optimize network depth, width, and image resolution simultaneously [5]. EfficientNetV2-B0 offers fast training convergence, parameter efficiency, and high feature extraction accuracy on edge and mobile deployments [5].
- **ResNet Architecture:** Employs residual skip connections to solve the vanishing gradient problem in deep networks, allowing effective feature learning across multi-scale visual patterns [8].

- **MobileNetV3-Large:** Integrates depthwise separable convolutions and squeeze-and-excitation modules to deliver lightweight inference for mobile devices, albeit sometimes at the cost of slight accuracy degradation on complex fine-grained classes [8].

2.2.4 Explainable AI (XAI) and Grad-CAM Principles

Standard deep learning classifiers function as opaque "black boxes," outputting probability scores without revealing the visual rationale behind their predictions [5] [8]. In clinical healthcare and quality-critical supply chains, this opacity creates a severe trust barrier among practitioners and quality managers.

Gradient-weighted Class Activation Mapping (Grad-CAM) resolves this opacity by generating coarse visual localization heatmaps [8] [10]. Grad-CAM calculates the gradients of the score for a target class C with respect to the feature map activations A^k of the final convolutional layer [8] [10]. These gradients are global-average-pooled to compute importance weights α^c_k:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial Y^c}{\partial A_{i,j}^k}$$

A weighted combination of forward activation maps is then processed through a Rectified Linear Unit (ReLU) function to highlight only those image regions that positively contribute to the target category [10] [8]:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right)$$

Superimposing these heatmaps onto input leaf images provides verifiable visual evidence—such as highlighting necrotic lesions or discoloration patterns—allowing

users to verify whether the model is focusing on true biological indicators rather than background artifacts [8] [10].

2.3 Related research

2.3.1 Automated Species Identification of Medicinal Plants

Numerous studies have explored deep learning for medicinal plant species classification [2] [5]. Lankasena et al. (2024) addressed species misidentification among morphologically similar medicinal plants in Sri Lanka, evaluating an EfficientNet-B0 model across 5 species of the genus Bauhinia and achieving a validation accuracy of 96.16% [5]. Jayanka and Fernando (2020) implemented a VGG16 architecture to recognize 17 Sri Lankan Ayurvedic species, demonstrating superior performance over traditional multi-layer perceptrons [5]. Gamage et al. (2024) developed a multilingual mobile platform incorporating a VGG16 classifier (96.88% accuracy) paired with crowd-sourced social media integration [2].

2.3.2 Plant Disease and Visual Quality Detection

Extending beyond species identification, researchers have investigated visual health condition screening [10]. Hoque et al. (2025) proposed an explainable, lightweight CNN framework for potato leaf disease detection, achieving 99.14% classification accuracy across early blight, late blight, and healthy states [10]. Zulkifli et al. (2025) developed a CNN-based lemon quality detection system through peel analysis to grade fruit condition. Similarly, Haque et al. (2022) and Brahimi et al. (2017) demonstrated that deep transfer learning architectures effectively categorize complex lesion patterns across agricultural crops [13].

2.3.3 Explainable AI Applications in Agricultural Diagnostics

The integration of XAI in agricultural computer vision has gained rapid momentum [8] [10] [13]. Gopalan et al. (2025) applied ResNet152 paired with Grad-CAM to diagnose corn leaf diseases, emphasizing that agricultural stakeholders require visual justification before acting on AI predictions [9]. Yadav et al. introduced a hybrid ResNet50 and EfficientNetB0 framework for Indian medicinal plant classification,

embedding Grad-CAM visualization and an explicit "Unknown" class to reject out-of-distribution inputs, reaching a validation accuracy of 92.96% [8].

2.4 Existing systems products and techniques

2.4.1 Global General-Purpose Identification Tools

Global applications such as **Pl@ntNet, PlantSnap, and FloraIncognita** provide large-scale species recognition across tens of thousands of global botanical species [2]. While highly effective for general botanical exploration, these platforms present distinct limitations for Ayurvedic supply chains:

- They lack specialized training on endemic Sri Lankan Ayurvedic flora.
- They do not evaluate leaf condition (disease/damage) or developmental maturity stages.
- They provide no visual explainability or medicinal suitability domain guidance.

2.4.2 Local and Domain-Specific Herbal Applications

Several domain-specific platforms have been introduced in Sri Lanka.

- **Arogya Platform (Pathiranage et al.):** Combines a VGG16 classifier (99.53% accuracy) with a Marker-based Watershed image segmentation algorithm to identify local herbal species [7].
- **Deeghayu (Nadeeshan et al. / Senevirathne et al. 2020):** A mobile assistive tool incorporating a CNN classification engine (92.5% accuracy) alongside 3D Augmented Reality (AR) plant visualization to assist herbal identification [2] [11].
- **AyurLeaf (Dileep & Pournami 2019):** A deep learning approach focused on classifying Indian and Sri Lankan medicinal leaves [2] [7].

2.4.3 Handcrafted Feature Extraction vs. End-to-End Deep Learning

Traditional systems required complex pre-processing chains, including HSV color masking, thresholding, and watershed contour detection before classifying statistical texture descriptors [1] [7]. In contrast, modern end-to-end transfer learning pipelines

accept letterboxed RGB leaf images, internally learning spatial feature representations while providing post-hoc visual interpretability through feature map gradient backpropagation [10] [8].

2.5 Gap synthesis and conceptual framework

2.5.1 Summary Matrix of Related Works

Study / System	Core Focus	Methodology / Backbone	Reported Accuracy	XAI / Explainability	Primary Limitation
Lankasena et al. (2024)	Bauhinia species confusion	EfficientNet-B0	96.16%	None	Focuses only on 5 species; no condition/maturity screening
Jayanka & Fernando (2020)	17 Sri Lankan Ayurvedic species	VGG16 CNN	High (Superior to MLP)	None	Single-stage identification only; black-box predictions
Pathiranage et al. (Arogya)	Herb identification & segmentation	VGG16 + Watershed Algorithm	99.53%	None	No health condition or maturity analysis; high computational footprint
Senevirathne et al. (Deeghayu)	Assistive learning tool	CNN + 3D Augmented Reality	92.50%	AR 3D Models (No Grad-CAM)	Focuses on educational AR visualization rather than quality assessment
Gamage et al. (2024)	Multilingual herb identification	VGG16 + Social Crowdsourcing	96.88%	None	Lacks visual quality screening and explainability features
Yadav et al. (2024)	Indian medicinal plant detection	ResNet50 + EfficientNetB0 Hybrid	92.96%	Grad-CAM Heatmaps	Evaluates species identity; does not perform species-conditioned condition/maturity
Hoque et al. (2025)	Potato leaf disease classification	Lightweight Custom CNN	99.14%	Grad-CAM Heatmaps	Restricted to agricultural crop diseases (potato);

					not applicable to Ayurvedic flora
Gopalan et al. (2025)	Corn leaf disease diagnosis	ResNet152	High	Grad-CAM Heatmaps	Single crop species focus; no knowledge base integration

Table 1: Comparative Analysis of Existing Research on Medicinal Plant Identification and Quality Assessment

2.5.2 Synthesized Research Gaps

Synthesizing the literature reveals four critical research gaps in automated medicinal plant processing:

1. **Lack of Interpretability in Quality Screening:** The vast majority of high-accuracy vision systems remain opaque "black boxes," providing classification labels without visual evidence showing which leaf regions drove the prediction [5] [10].
2. **Absence of Species-Conditioned Multi-Stage Architectures:** Existing disease classifiers evaluate general crop conditions without conditioning diagnostic heads on species identity, leading to poor granularity and invalid condition predictions for unstudied species.
3. **Inadequate Out-of-Distribution Handling:** Standard models force every input into one of the trained classes, lacking rejection thresholds or *Unknown* classes to safely handle unsupported plant species or background noise [5] [8].
4. **Disconnect Between Computer Vision and Domain Knowledge Bases:** Current tools output raw technical labels (e.g., "Leaf Spot") without linking predictions to structured Ayurvedic knowledge bases detailing observable symptoms, harvesting guidelines, and medicinal suitability categories.

2.5.3 Conceptual Framework of the Proposed System

- To address these combined research gaps, this dissertation proposes a **hierarchical, species-aware explainable AI decision-support framework.** As illustrated in the conceptual workflow below, the system structures quality evaluation across three sequential processing tiers:

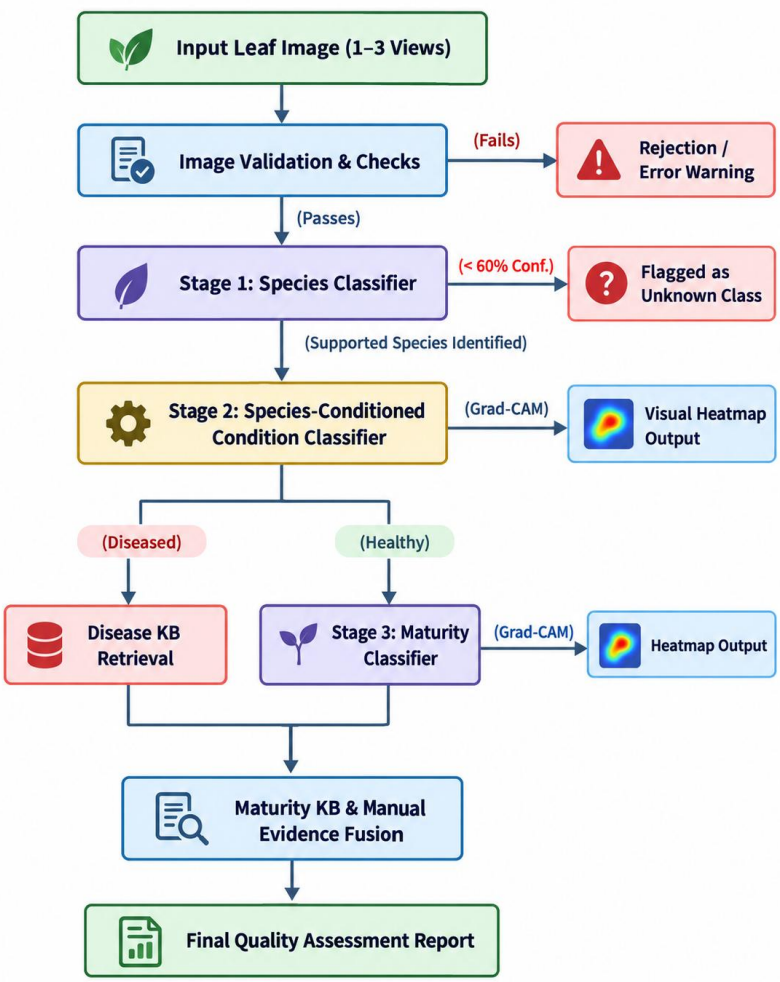


Figure 1: Conceptual Framework of the Proposed Ayurvedic Medicinal Plant Quality Assessment System

Pre-Inference Validation & Thresholding: Input images undergo automated resolution, blur, and brightness verification before inference, filtering out unprocessable samples.

- **Stage 1 (Species Identification with Unknown Rejection):** An EfficientNetV2-B0 model classifies species identity. Predictions falling

below a 60% softmax confidence threshold are routed to an *Unknown* class, rejecting unsupported inputs.

- **Stage 2 (Species-Conditioned Health Classification):** Inputs are passed to a shared CNN backbone equipped with species-specific output heads, classifying visible leaf health conditions (healthy, diseased, pest-damaged, leaf rot) tailored to that exact plant. Grad-CAM backpropagates gradients to generate an interactive visual heatmap overlaying lesion areas [8].
- **Stage 3 (Conditional Maturity & Knowledge Fusion):** If the leaf is classified as healthy and belongs to a maturity-supported species, Stage 3 assesses developmental stage (young vs. mature). Predictions link directly to CSV-based domain knowledge bases, translating visual outputs into structured descriptions, harvesting advice, and medicinal suitability categories.

3 RESEARCH METHODOLOGY

3.1 Research design

This research employs an experimental, quantitative software engineering design to develop and evaluate an **Explainable Artificial Intelligence (XAI) decision-support system** for Ayurvedic medicinal plant quality assessment. The methodology models quality screening as a **hierarchical, three-stage deep learning workflow**:

- **Stage 1 (Species Identification & Rejection):** Accepts input images, verifies species identity among supported Ayurvedic plants, and routes low-confidence or out-of-distribution inputs to an *Unknown* rejection class.
- **Stage 2 (Species-Conditioned Health Assessment):** Evaluates visible leaf health conditions using specialized species-specific output heads driven by a shared feature extraction backbone.
- **Stage 3 (Conditional Maturity Classification):** Assesses developmental maturity stages for healthy leaves across supported species.

The research design pairs deep transfer learning architectures with **Gradient-weighted Class Activation Mapping (Grad-CAM)** to supply visual explanations for model predictions, alongside automated image quality checks, multi-image probability aggregation, and domain knowledge base retrieval.

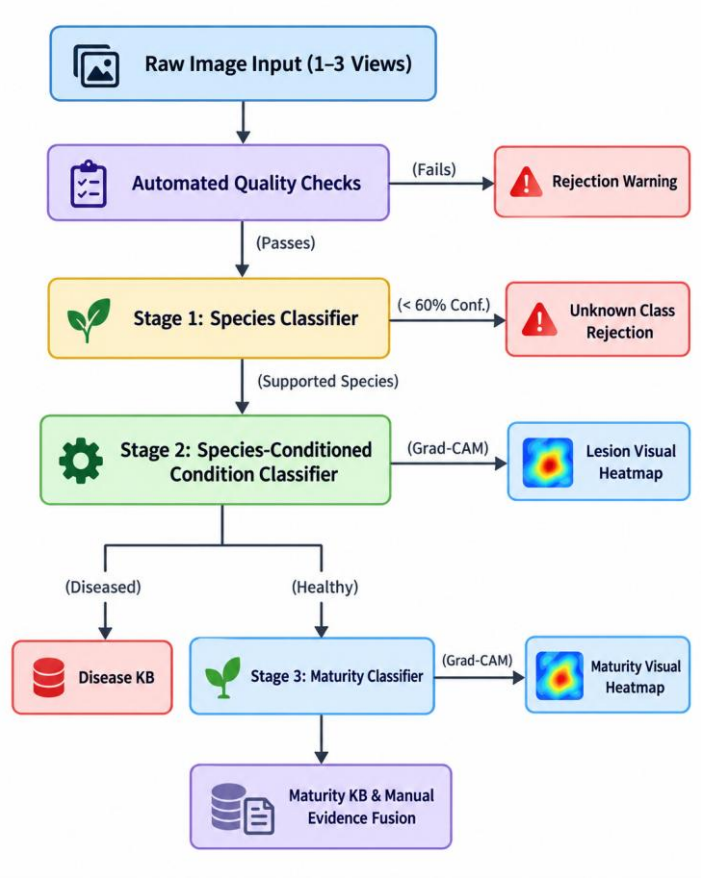


Figure 2: Research Methodology and Three-Stage Classification Workflow

3.2 Data participants or study materials

3.2.1 Dataset Composition and Stage Distribution

The empirical evaluation relies on three curated datasets covering six key Sri Lankan Ayurvedic species: Betel (*Piper betle*), Centella asiatica (*Gotu Kola*), Holy Basil (*Tulsi*), Kalanchoe pinnata (*Akkapana*), Murraya koenigii (*Curry leaf / Karapincha*), and Neem (*Kohomba*), alongside an explicit *Unknown* background class.

- **Stage 1 (Species Dataset):** Consists of **10,534 images** across 7 classes (6 supported species + *Unknown*). Data partitioning follows a **70% training (7,370), 15% validation (1,578), and 15% testing (1,586)** distribution using stratified random sampling (random seed 42).

- **Stage 2 (Condition Dataset):** Comprises **13,521 total images** across 36 species-conditioned health classes (Training: 9,089 | Validation: 2,187 | Test: 2,245).
- **Stage 3 (Maturity Dataset):** Contains **848 images** across 5 species that visually support maturity differentiation (Training: 620 | Validation: 107 | Test: 121; Holy Basil is excluded from maturity screening).

Stage	Total Images	Training	Validation	Test Set
Stage 1 Species	10,534	7,370	1,578	1,586
Stage 2 Condition	13,521	9,089	2,187	2,245
Stage 3 Maturity	848	620	107	121

Table 2: Dataset Distribution Across Training, Validation, and Testing Sets for the Three Classification Stages

Detailed Class Distribution Summary:

- **Stage 1 Species Classes:** Betel (1,350), Centella asiatica (1,924), Holy Basil (687), Kalanchoe pinnata (867), Murraya koenigii (763), Neem (1,583), Unknown (3,360).
- **Stage 2 Condition Classes (36 Heads):** Betel (3 classes: Healthy, Leaf_Rot, Leaf_Spot), Centella (5 classes: Dried, Healthy, Phosphorus_deficiency, Redmite, WormCreep), Holy Basil (5 classes: Anthocyanin_Pigmentation, Downy_Mildew, Healthy, Web_Blight, Yellow_Spot), Kalanchoe (5 classes: Healthy, Insect_Damage, Web_Blight, Yellow, Yellow_Blight), Murraya (4 classes: Complex_Foliar_Damage, Fungus, Healthy, Insect_Damage), Neem (14 classes: Alternaria, Bacterial_infection, Defoliator, Dieback, Healthy, Irregular_yellowing, Leaf_Blight, Leaf_Miners,

Leaf_Miners_Powdery_Mildew, Leaf_Rust, Powdery_Mildew, Spot, Web_Blight, Yellow).

- **Stage 3 Maturity Classes (11 Heads):** Betel (Mature, Young), Centella (Mature, Young), Kalanchoe (Mature, Young), Murraya (Developing, Mature, Young), Neem (Mature, Young).

3.2.2 Image Preprocessing & Aspect-Preserving Letterboxing

To maintain structural consistency without introducing aspect ratio distortion:

1. **OpenCV BGR to RGB Conversion:** Decodes raw image input bytes into standard RGB color space.
2. **Aspect-Preserving Resize:** Rescales leaf images while preserving native proportions using bilinear/interpolation methods.
3. **Black Letterboxing:** Symmetric zero-padding is applied to pad images into a uniform $224 \times 224 \times 3$ tensor format.
4. **Channel Normalization:** Scales RGB values (0–255) using ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]):

$$\mathbf{x}_{\text{norm}} = \frac{\frac{\mathbf{x}}{255} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

3.2.3 Data Augmentation Strategies

- **Offline Minority Augmentation:** Generated additional samples for under-represented classes (e.g., Centella Phosphorus deficiency, Neem Leaf Rust) using random horizontal flips ($p = 0.5$) spatial rotations ($\pm 14^{\circ}$ to $\pm 18^{\circ}$), scaling (0.92–1.15), Gaussian noise ($\sigma = 2.5 - 3.0, p = 0.20 - 0.25$), and Gaussian blur ($3 \times 3, p = 0.12 - 0.15$)
- **Online Train-Time Augmentation:** Implemented in TensorFlow input pipelines to prevent overfitting:
    - Random Horizontal Flip (p=0.5)
    - Random Rotation (factor 0.08)

- Random Zoom (factor 0.10–0.12)
- Random Contrast Adjustment (factor 0.15)

3.3 Experimental or study procedure

3.3.1 Model Architectures and Transfer Learning Configurations

The primary system configuration (**EEE**) deploys **EfficientNetV2-B0** transfer learning backbones pre-trained on ImageNet across all three pipeline stages.

1. **Stage 1 (Species Classifier):** Frozen EfficientNetV2-B0 backbone → Global Average Pooling → Dropout (0.25) → Dense 7-class Softmax layer.
2. **Stage 2 (Species-Conditioned Health Classifier):** Shared EfficientNetV2-B0 backbone equipped with **six specialized softmax output heads** (Betel: 3, Centella: 5, Holy Basil: 5, Kalanchoe: 5, Murraya: 4, Neem: 14). Inputs are routed dynamically based on Stage 1 species predictions.
3. **Stage 3 (Maturity Classifier):** Shared EfficientNetV2-B0 backbone with **five specialized softmax output heads** (Betel: 2, Centella: 2, Kalanchoe: 2, Murraya: 3, Neem: 2).

3.3.2 Model Training Hyperparameters

- **Optimizer:** Adam ($lr = 0.001, \beta_1 = 0.9, \beta_2 = 0.999$)
- **Batch Size:** 32 (Stage 1) / 16 (Stages 2 & 3).
- **Epochs:** 15–20 epochs with early stopping (patience 5 based on validation loss).
- **Loss Functions:**
    - Stage 1: Categorical Cross-Entropy.
    - Stages 2 & 3: Masked Sparse Categorical Cross-Entropy across active output heads:

$$\mathcal{L}_{\text{Condition}} = \sum_{h=1}^{6} m_h \cdot \text{CCE}(y_h, \hat{y}_h)$$

3.3.3 Pre-Inference Validation and Threshold Routing

1. **Automated Quality Checks:** Filters out unprocessable images based on resolution($\geq 224 \times 224$), blur (Laplacian variance ≥ 20), and brightness (mean grayscale value $35 \leq$ brightness ≤ 220).
2. **Species Confidence Gate** ($\tau_{\text{species}} = 0.60$**):** If max P(species) < 0.60 or the predicted category is *Unknown*, execution halts and flags the input as unsupported.
3. **Maturity Execution Gate:** Stage 3 executes **only** if Stage 2 classifies the leaf as *Healthy*. Non-healthy leaves directly trigger disease knowledge base lookup.

3.3.4 Multi-Image Probability Aggregation

When users capture *N* images ($1 \leq N \leq 3$) of a sample, the pipeline performs **multi-view probability averaging** across individual softmax outputs:

$$\bar{p}(c) = \frac{1}{N}\sum_{i=1}^{N} p_i\,(c)$$

The final prediction corresponds to $\arg\max_c \bar{p}(c)$ with overall confidence defined as $\max_c \bar{p}(c)$

3.3.5 Grad-CAM Visual Explainability Setup

Grad-CAM backpropagates predicted class gradients to the final convolutional feature maps (top_activation layer of EfficientNetV2-B0):

1. Neuron Importance Weights:

$$\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial y^c}{\partial A_{i,j}^k}$$

2. Localization Map Generation:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c\, A^k\right)$$

3.4 Measures and evaluation metrics

System performance is measured across statistical classification metrics, operational efficiency parameters, and visual explainability quality:

- **Classification Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision, Recall, and F1-Score:**

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Macro and Weighted F1-Scores:** Evaluates performance across balanced and imbalanced class distributions.
- **Operational Latency & Footprint:** Measured in inference execution time per image (milliseconds) and model checkpoint disk size (megabytes).
- **Visual Explanation Verification:** Qualitative verification of Grad-CAM heatmaps over known biological lesion regions.

3.5 Data analysis

Data analysis evaluates performance across isolated pipeline stages and end-to-end integration:

1. **Isolated Stage Evaluation:** Generates confusion matrices and per-class precision/recall metrics for Stage 1 (Species), Stage 2 (Condition), and Stage 3 (Maturity).
2. **Backbone Benchmarking:** Compares EfficientNetV2-B0 against MobileNetV3-Large and ResNet50 baselines across accuracy, latency, and memory footprint.
3. **Multi-Image Aggregation Analysis:** Evaluates how averaging softmax probabilities across 1 to 3 images affects classification stability and reduces single-view variance.

4. **Rejection & Error Analysis:** Examines model behavior on out-of-distribution inputs, low-confidence samples, and misclassified biological conditions.

3.6 Ethics privacy security and responsible AI

- **Data Minimization & Privacy:** The dataset consists strictly of botanical leaf images. No personally identifiable information (PII), facial biometric data, or sensitive user records are collected or processed.
- **Responsible AI & Safety Scope:** The system explicitly frames its outputs as visual quality screening. It does **not** claim to perform chemical assay analysis, active alkaloid/phytochemical profiling, or biological toxicity testing.
- **Rejection Safeguards:** Enforcing a 60% species confidence threshold and an *Unknown* class prevents the system from forcing invalid predictions on unsupported plant species or corrupted inputs.
- **Visual Transparency:** Integrating Grad-CAM heatmaps ensures non-expert users can visually inspect and verify the reasoning behind AI quality ratings before acting on recommendations.

3.7 Reproducibility and research artefacts

To ensure complete experimental reproducibility, all research artefacts are structured and documented within the project repository:

- **Source Code Repository:** Maintained under Git version control (branch dev, commit 9c2d0aa6018f...).
- **Dataset CSV Manifests:** Complete sample tracking files (datasets_species.csv, datasets_condition.csv, datasets_maturity.csv).

- **Metric Logs & Confusion Matrices:** Archived training histories and evaluation records (model_metrics.csv, model_comparison.csv, training_histories.csv).
- **Model Checkpoints:** Saved TensorFlow .h5 model files representing pre-trained backbones and species-conditioned output heads.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Functional Requirements

The functional requirements define the core operational capabilities of the **Explainable AI System for Ayurvedic Medicinal Plant Quality Assessment**. The system is designed to automate visual leaf screening, enforce confidence safeguards, generate visual explanations, and translate model predictions into clinical domain recommendations.

Requirement ID	Functional Requirement	Description	System Module
FR1	Automated Image Validation	The system shall evaluate uploaded leaf images for spatial resolution ($\geq 224 \times 224$), blur (Laplacian variance≥ 20), and illumination ($35 \leq$ brightness ≤ 220) prior to model execution	image_validator.py
FR2	Species Identification & OOD Rejection	The system shall classify input images across 6 supported Ayurvedic species and enforce a 60% confidence threshold ($\tau_{\text{species}} = 0.60$) to reject out-of-distribution or unknown inputs	plant_identifier.py
FR3	Species-Conditioned Health Classification	The system shall route verified species inputs to specialized output heads on a shared backbone to classify 36 species-specific condition categories	condition_classifier.py
FR4	Conditional Maturity Assessment	The system shall execute maturity classification **only** for healthy leaves of maturity-supported species (Betel, Centella, Kalanchoe, Murraya,	maturity_classifier.py

		Neem) while bypassing unsupported species	
FR5	Grad-CAM Visual Explanation Generation	The system shall generate Gradient-weighted Class Activation Mapping heatmaps overlaid on original RGB images for the active classification stage	gradcam.py
FR6	Domain Knowledge Base Retrieval	The system shall map predicted disease conditions and maturity stages to structured CSV knowledge bases supplying clinical descriptions, symptoms, and medicinal suitability categories	disease_lookup.py, maturity_lookup.py
FR7	Multi-Image Probability Aggregation	The system shall accept 1 to 3 leaf image views and average their softmax probability distributions to deliver a stable sample-level classification	api.py
FR8	Manual Evidence Fusion (Backend)	The system shall accept optional user-measured leaf physical dimensions (length, width in cm) and surface traits to adjust maturity support scores	manual_maturity_support.py

Table 3: Functional Requirements of the Ayurvedic Medicinal Plant Quality Assessment System

4.1.2 Non-Functional Requirements

The non-functional requirements establish performance benchmarks, security guidelines, and architectural quality standards:

- **NFR1: Classification Reliability & Calibration:** The system must maintain high classification accuracy across all pipeline stages while ensuring softmax probabilities accurately reflect prediction confidence. Rejection gates must prevent false predictions on unstudied species.

- **NFR2: Visual Interpretability & Trust:** Visual Grad-CAM heatmaps must accurately localize biological features (such as necrotic lesions, fungal spots, or chlorosis patterns) to allow non-expert users to visually verify model reasoning.
- **NFR3: Real-Time API Responsiveness:** The REST backend must process image validation, species identification, condition routing, maturity assessment, and Grad-CAM generation within interactive request timeouts (25s, 30s, and 35s send thresholds).
- **NFR4: Modular Microservice Architecture:** The quality assessment engine must function as an independent, decoupled microservice within the broader group platform, exposing well-defined RESTful endpoints.
- **NFR5: Lightweight Resource Footprint:** Neural network models must be optimized for server deployment, maintaining a parameter footprint suitable for constrained cloud hosting environments.
- **NFR6: Safety & Ethical Advisory Scope:** System outputs must explicitly present quality assessments as advisory screening tools, clarifying that visual processing does not substitute for chemical alkaloid assay analysis or biological toxicity testing.

4.2 Component architecture within the complete solution

4.2.1 Overall Platform Integration Context

This component—**"Explainable AI System for Ayurvedic Medicinal Plant Quality Assessment"**—serves as Pillar 2 of the integrated platform **"Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis"**[1][16]. The platform combines four distinct functional modules:

1. **Pillar 1:** Automated Medicinal Plant Species Identification.
2. **Pillar 2 (This Component):** Explainable Quality Assessment, Health Condition Screening, Maturity Classification, and Grad-CAM Visual Explainability.

3. **Pillar 3:** Conversational Ayurvedic Knowledge Retrieval & RAG Chatbot.
4. **Pillar 4:** Agricultural Cultivation Advisory & Market Price Forecasting.

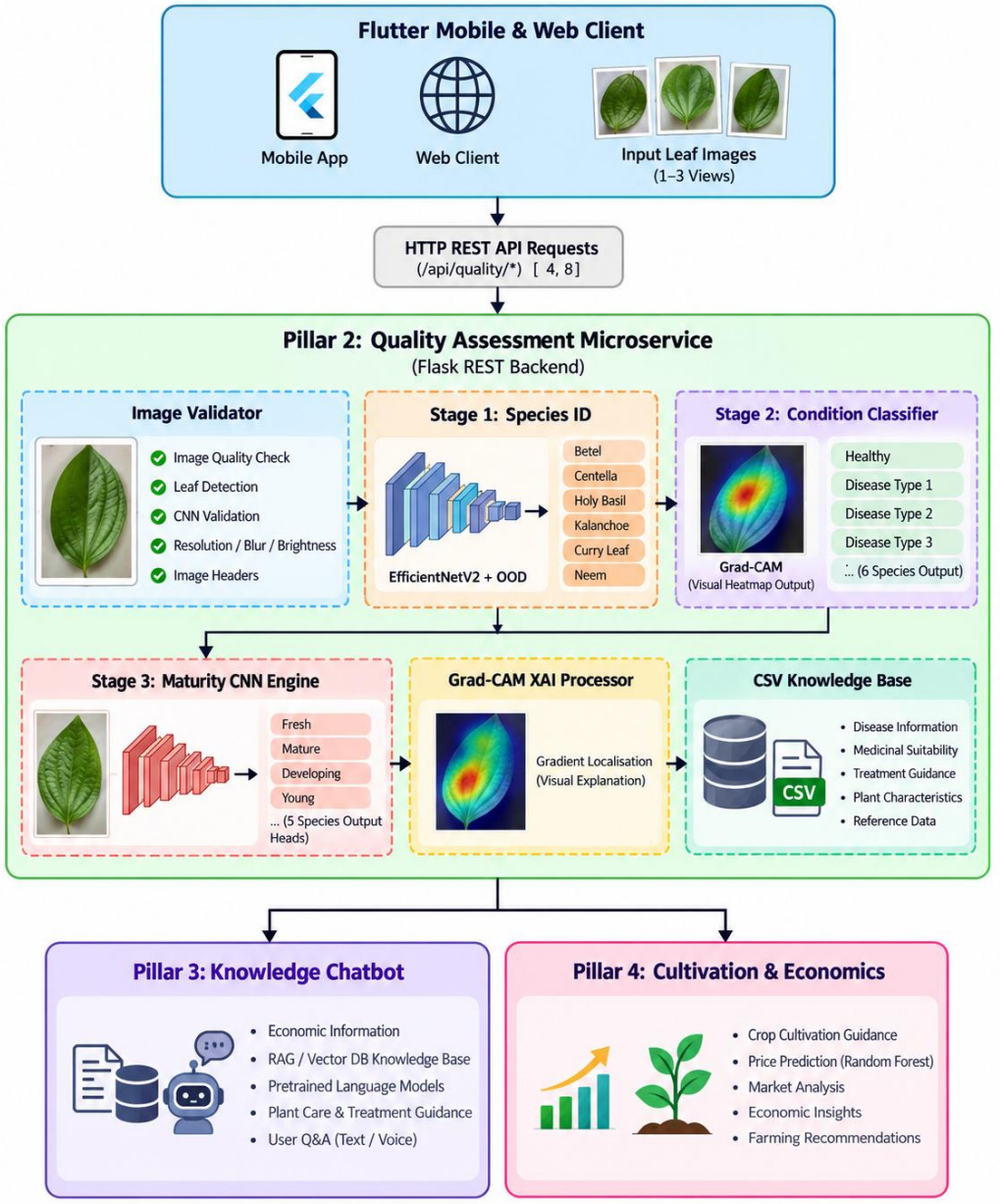


Figure 3: Overall Platform Architecture and Integration of the Quality Assessment Component

The quality assessment module communicates directly with the Flutter client over HTTP JSON routes. If a user requests additional background information

regarding a detected disease, the Flutter UI gracefully transitions context to Pillar 3's General Public Chatbot interface.

4.2.2 High-Level Three-Tier Component Architecture

The component implements a modular **3-Tier Software Architecture** encompassing Presentation, Application Logic, and Data/AI layers:

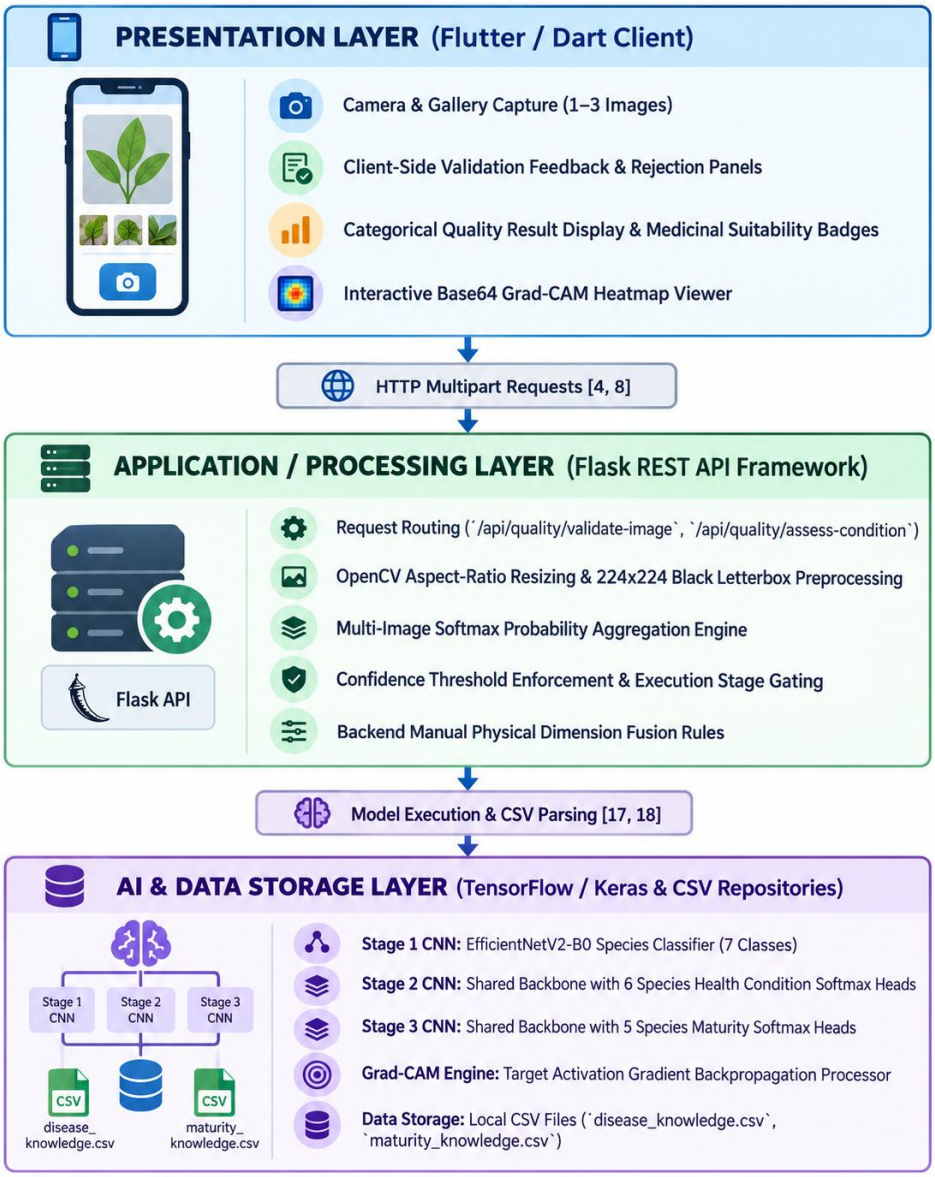


Figure 4 : Three-Tier Architecture of the Quality Assessment System

4.2.3 End-to-End Execution Flow

The sequence diagram below illustrates the exact hierarchical execution path triggered when a user submits images for quality evaluation:

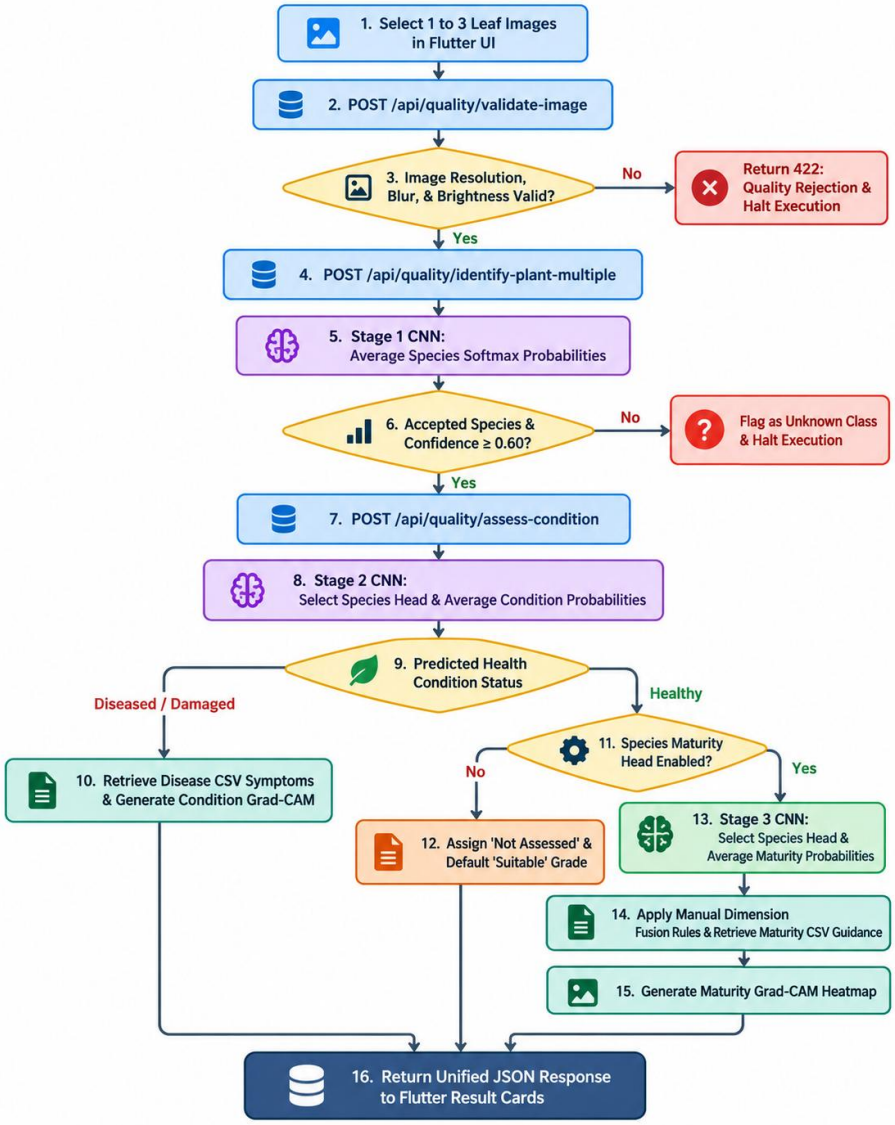


Figure 5: End-to-End Execution Flow of the Medicinal Plant Quality Assessment Pipeline

4.3 Design alternatives and decisions

During architectural design, several key engineering trade-offs were evaluated to establish an optimal balance between accuracy, interpretability, and operational

latency.

4.3.1 Single-Stage All-in-One Classifier vs. Hierarchical Multi-Stage Pipeline

- **Option A (Single-Stage Classifier):** Train a single monolithic CNN to classify all species, diseases, and maturity levels in one single step (36 condition classes + 11 maturity classes + species = over 50 combined outputs).
- **Option B (Hierarchical Multi-Stage Pipeline - Selected):** Separate the pipeline into Stage 1 (Species Selection), Stage 2 (Species-Conditioned Health Classification), and Stage 3 (Healthy-Branch Maturity Assessment).
- **Trade-off Analysis & Decision:** Option A suffers severe class imbalance, cross-species feature confusion, and high vulnerability to out-of-distribution inputs. Option B enforces logical domain constraints—routing images only to valid species-specific diagnostic heads, isolating disease features from maturity traits, and enabling targeted rejection thresholds at early processing **stages.**

Design Dimension	Single-Stage Monolithic CNN	Hierarchical Multi-Stage Pipeline
Class Complexity	>50 combined unconditioned classes	Isolated stages (7 species, 36 condition classes, 11 maturity classes)
Cross-Species Leak	**High:** Neem blight features can confuse Betel rot categories	**Zero:** Species identification locks the active diagnostic head
Out-of-Distribution	**Poor:** Forces non-medicinal leaves into arbitrary disease classes	**Robust:** Stage 1 rejects inputs below 60% confidence
Execution Logic	Evaluates maturity even on rotting leaves	**Gated:** Stage 3 runs only on healthy leaves

Figure 6: Comparison of Single-Stage and Hierarchical Multi-Stage Classification Architectures

4.3.2 Backbone Model Selection: EfficientNetV2-B0 vs. MobileNetV3-Large vs. ResNet50

- Option A (ResNet50): Deep residual skip-connection network pre-trained on ImageNet.
- Option B (MobileNetV3-Large): Lightweight mobile network employing depthwise separable convolutions.
- Option C (EfficientNetV2-B0 - Selected): Compound-scaled neural architecture utilizing fused-MBConv blocks.
- Trade-off Analysis & Decision: ResNet50 exhibits a heavy parameter footprint (~98 MB checkpoint size), increasing cloud deployment memory costs. MobileNetV3-Large provides fast execution but exhibits slight accuracy degradation on fine-grained lesion spots. EfficientNetV2-B0 delivers superior feature extraction accuracy with a parameter footprint of ~5.9M (~22–24 MB checkpoint size), making it the optimal backbone for multi-stage deployment.

4.3.3 Multi-Head Shared Backbone vs. Independent Model Files

- **Option A (Independent Per-Species CNN Files):** Train and load separate, standalone CNN model files for each individual plant species (6 condition models + 5 maturity models = 11 separate files).
- **Option B (Shared Backbone with Species Output Heads - Selected):** Deploy a single shared EfficientNetV2-B0 feature extractor paired with specialized softmax classification heads for each species.
- **Trade-off Analysis & Decision:** Option A requires loading multiple heavy model files into backend memory, causing severe RAM exhaustion and slow cold-start initialization times. Option B shares low-level convolutional feature representations across all species, dramatically reducing memory usage while allowing target heads to learn species-specific diagnostic distributions.

4.3.4 Explainability Approach: Grad-CAM vs. LIME vs. SHAP

- **Option A (LIME / SHAP):** Perturbation-based model-agnostic explainability methods.

- **Option B (Grad-CAM - Selected):** Gradient-weighted Class Activation Mapping backpropagating prediction gradients to final convolutional layer feature maps.
- **Trade-off Analysis & Decision:** LIME and SHAP require generating hundreds of image perturbations per prediction, resulting in prohibitive inference latencies (5–15 seconds per image). Grad-CAM computes spatial activation maps in a single backpropagation pass (~100–150 ms execution time), yielding real-time, high-resolution visual heatmaps ideal for interactive mobile applications.

4.4 Detailed design

4.4.1 Data Preprocessing & Input Standardization Pipeline

To maintain input spatial consistency across diverse client mobile cameras, all incoming leaf images pass through an aspect-preserving transformation pipeline prior to neural network inference:

1. **Decoding:** OpenCV decodes raw request bytes into a 3-channel RGB image tensor.
2. **Aspect-Preserving Rescale:** Image proportions are scaled down such that the maximum dimension equals 224 pixels while maintaining native aspect ratios.
3. **Centered Black Letterboxing:** Symmetrical zero-padding (black borders) is applied around the scaled image to construct a standardized $224 \times 224 \times 3$ tensor format.
4. **Channel Normalization:** Pixel intensity values (0–255) are normalized using ImageNet channel mean ($\boldsymbol{\mu} = [0.485, 0.456, 0.406]$) and standard deviation ($\boldsymbol{\sigma} = [0.229, 0.224, 0.225]$) vectors embedded directly within TensorFlow model graphs:

$$\mathbf{x}_{\text{norm}} = \frac{\frac{\mathbf{x}}{255} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

4.4.2 Multi-Image Probability Aggregation Algorithm

When users capture N images ($1 \leq N \leq 3$) of a single plant sample, the API averages individual softmax class probabilities across all valid views:

$$\bar{p}(c) = \frac{1}{N}\sum_{i=1}^{N} p_i\,(c)$$

The aggregated predicted category corresponds to $\hat{c} = \arg\max_{c} \bar{p}(c)$, with sample-level classification confidence defined as $\text{Conf} = \max_{c} \bar{p}(c)$

4.4.3 REST API Contracts & Schema Definitions

Communication between the Flutter frontend and Flask backend is governed by four primary POST multipart/form-data HTTP endpoints

1. POST /api/quality/validate-image

- **Request:** Single image file upload (image).
- **Response (200 OK / 422 Unprocessable):**


```
{
  "valid": true,
  "reason": null,
  "resolution": {"width": 1080, "height": 1440},
  "blur_score": 142.6,
  "brightness": 118.4,
  "warnings": []
}
```


2. POST /api/quality/identify-plant-multiple

- **Request:** 1 to 3 image files (images).
- **Response (200 OK / 422 Rejected):**


```
{
  "accepted": true,
```

"species": "Betel",
"scientific_name": "Piper betle",
"confidence": 0.942,
"image_count": 2,
"rejection_reason": null
}

3. POST /api/quality/assess-condition

- **Request:** 1 to 3 image files (images) and optional manual_inputs JSON string.
- **Response (200 OK):**

{
"accepted": true,
"plant": {
"species": "Betel",
"scientific_name": "Piper betle",
"confidence": 0.942
},
"condition": {
"status": "Diseased",
"label": "Leaf_Spot",
"confidence": 0.885,
"description": "Fungal leaf spot caused by Colletotrichum piperis."
},
"disease_info": {

```
    "symptoms": "Circular black or dark brown spots with yellow halos.",
    "action": "Isolate affected leaves; apply copper-based organic fungicide."
  },
  "maturity": null,
  "medicinal_suitability": {
    "category": "Unsuitable",
    "rationale": "Leaf exhibits active fungal lesion spots."
  },
  "xai": {
    "stage": "condition",
    "heatmap_base64": "..."
  }
}
```


4.4.4 Domain Knowledge Base Schema

Model predictions trigger automated lookup queries against structured CSV knowledge bases:

1. Disease Knowledge Base Schema (quality.disease_knowledge.csv):

- species (String): Target plant species name (e.g., *Neem*).
- condition_label (String): Predicted health condition class (e.g., *Powdery_Mildew*).
- description (Text): Botanical definition of the pathology.
- symptoms (Text): Observable visual lesions and surface traits.
- recommended_action (Text): Agricultural handling and organic control guidance.

2. Maturity & Suitability Knowledge Base Schema (quality.maturity_knowledge_with_medicinal_suitability.csv):

- species (String): Target plant species name.
- maturity_stage (String): Developmental maturity label (*Young*, *Mature*, *Developing*).
- medicinal_suitability (String): Quality grade (*Suitable*, *Conditional*, *Unsuitable*).
- suitability_rationale (Text): Clinical explanation of phytochemical readiness.
- harvesting_advice (Text): Guidelines for optimal harvesting timing and preservation.

4.4.5 Grad-CAM Heatmap Generation Algorithm

The visual explainability engine backpropagates target class gradients through the neural network to highlight important spatial features [14]:

1. **Target Feature Map:** Extracts activations A^k from the final convolutional layer of EfficientNetV2-B0 (top_activation).
2. **Neuron Importance Weight Calculation:** Computes the global average pooling of gradients of target class score Y^c with respect to feature map A^k.

$$\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial y^c}{\partial A_{i,j}^k}$$

3. **Rectified Linear Map Generation:** Computes a weighted sum of forward feature maps passed through a ReLU activation to retain positive contributions:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right)$$

4. **Colormap & Superposition:** Heatmaps are rescaled to original image dimensions, colormapped using OpenCV JET, and blended with original RGB leaf images ($\alpha = 0.5, \beta = 0.5$) before converting to Base64 PNG strings for transport.

4.5 Standards and professional practice

The design and implementation of the quality assessment system adhere strictly to established software engineering, artificial intelligence, and healthcare technology standards:

4.5.1 Software Engineering & RESTful API Standards

- **RESTful Architecture Constraints:** API routes strictly follow REST standards, utilizing standard HTTP status codes (200 OK for valid executions, 400 Bad Request for invalid inputs, 413 Payload Too Large for files exceeding the 25 MiB upload cap, and 422 Unprocessable Entity for quality or confidence rejections).
- **JSON Schema Standardization:** All endpoint responses comply with rigid, predictable JSON schemas to guarantee seamless client parsing across mobile and web deployments.
- **Modular Microservice Separation:** Separation of concerns is maintained between presentation widgets, Flask route controllers, CNN inference engines, and CSV parsing modules.

4.5.2 Responsible AI, Safety, & Ethics Standards

- **Explainable AI (XAI) Compliance:** Integrating Grad-CAM heatmaps ensures the system provides verifiable visual evidence alongside classification scores, aligning with IEEE and EU AI Act principles for trustworthy AI.
- **Explicit Out-of-Distribution Safeguards:** Enforcing a 60% species confidence gate prevents the network from outputting arbitrary, over-confident predictions on unstudied plant species or non-leaf objects.
- **Advisory Medical Disclaimer:** To maintain ethical compliance, system outputs are strictly framed as visual screening tools, explicitly notifying users that automated visual quality assessment does not replace HPLC chemical assay testing or professional Ayurvedic laboratory analysis.

4.5.3 Code Quality & Repository Standards

- **PEP 8 Compliance:** Python backend scripts adhere to standard PEP 8 formatting, exception handling, and modular structure conventions.
- **Dart Clean Architecture:** Flutter client pages implement clean UI component separation, utilizing asynchronous futures, explicit request timeouts, and graceful network error handling.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The development and execution environment for the **Explainable AI System for Ayurvedic Medicinal Plant Quality Assessment** is structured as a decoupled, full-stack microservice architecture. The system combines a **Python Flask REST backend** hosting deep learning inference engines with a **Flutter (Dart) cross-platform mobile and web client.**



Figure 7: Development Environment and Runtime Technology Stack

5.2 Component implementation

5.2.1 Image Validation Module (quality.image_validator.py)

To prevent unprocessable or degraded captures from reaching the deep neural networks, incoming leaf images pass through an automated pre-inference quality screening pipeline:

```
import cv2
import numpy as np


def validate_leaf_image(image_bytes: bytes) -> dict:
    # Decode image bytes using OpenCV
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    if img is None:
        return {"valid": False, "reason": "INVALID_IMAGE"}

    h, w, _ = img.shape
    if h < 224 or w < 224:
        return {"valid": False, "reason": "LOW_RESOLUTION"}

    # Calculate Variance of Laplacian for blur detection
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur_score = cv2.Laplacian(gray, cv2.CV_64F).var()

if blur_score < 20.0:
    return {
        "valid": False,
        "reason": "IMAGE_TOO_BLURRY",
        "blur_score": blur_score
    }

# Calculate Mean Illumination Intensity
brightness = float(np.mean(gray))

if brightness < 35.0:
    return {
        "valid": False,
        "reason": "IMAGE_TOO_DARK",
        "brightness": brightness
    }

elif brightness > 220.0:
    return {
        "valid": False,
        "reason": "IMAGE_OVEREXPOSED",
        "brightness": brightness
```

```
    }


  return {
    "valid": True,
    "resolution": {
      "width": w,
      "height": h
    },
    "blur_score": blur_score,
    "brightness": brightness
  }
```

5.2.2 Stage 1 Species Identification & Out-of-Distribution Rejection (quality.plant_identifier.py)

Stage 1 classifies input leaf images across seven output categories: Betel, Centella asiatica, Holy Basil, Kalanchoe pinnata, Murraya koenigii, Neem, and *Unknown.*

```
import tensorflow as tf
import numpy as np
import cv2



class PlantIdentifier:
  def __init__(self, model_path: str, confidence_threshold: float = 0.60):
    self.model = tf.keras.models.load_model(model_path, compile=False)
    self.threshold = confidence_threshold
```

```
    self.classes = [
        'betel',
        'centella_asiatica',
        'holy_basil',
        'kalanchoe_pinnata',
        'murraya_koenigii',
        'neem',
        'unknown'
    ]

def preprocess_letterbox(self, img_bgr: np.ndarray) -> np.ndarray:
    # Aspect-preserving resize with black letterbox padding to 224x224x3
    h, w = img_bgr.shape[:2]
    scale = 224.0 / max(h, w)
    nh, nw = int(h * scale), int(w * scale)

    resized = cv2.resize(
        img_bgr,
        (nw, nh),
        interpolation=cv2.INTER_AREA
    )

    canvas = np.zeros((224, 224, 3), dtype=np.uint8)
    top = (224 - nh) // 2
    left = (224 - nw) // 2
```

```
        canvas[top:top+nh, left:left+nw] = resized

        # BGR to RGB conversion and Float32 scaling
        img_rgb = cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB)
        return np.expand_dims(img_rgb.astype(np.float32), axis=0)

    def predict_aggregated(self, image_bytes_list: list) -> dict:
        probs_list = []

        for img_bytes in image_bytes_list:
            nparr = np.frombuffer(img_bytes, np.uint8)
            img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

            if img is None:
                raise ValueError("Invalid image data")

            tensor = self.preprocess_letterbox(img)
            preds = self.model.predict(tensor, verbose=0)

            probs_list.append(preds[0])

        # Average softmax probabilities across 1 to 3 leaf views
        avg_probs = np.mean(probs_list, axis=0)
        top_idx = int(np.argmax(avg_probs))
```

```
top_prob = float(avg_probs[top_idx])
predicted_class = self.classes[top_idx]

# Out-of-Distribution Rejection Gate
if top_prob < self.threshold or predicted_class == 'unknown':
    return {
        "accepted": False,
        "species": "Unknown",
        "confidence": top_prob,
        "reason": "UNKNOWN_OR_LOW_CONFIDENCE"
    }

return {
    "accepted": True,
    "species": predicted_class,
    "confidence": top_prob
}
```

5.2.3 Stage 2 Species-Conditioned Health Classifier (quality.condition_classifier.py)

Stage 2 utilizes a shared EfficientNetV2-B0 backbone equipped with **six specialized softmax output heads**:

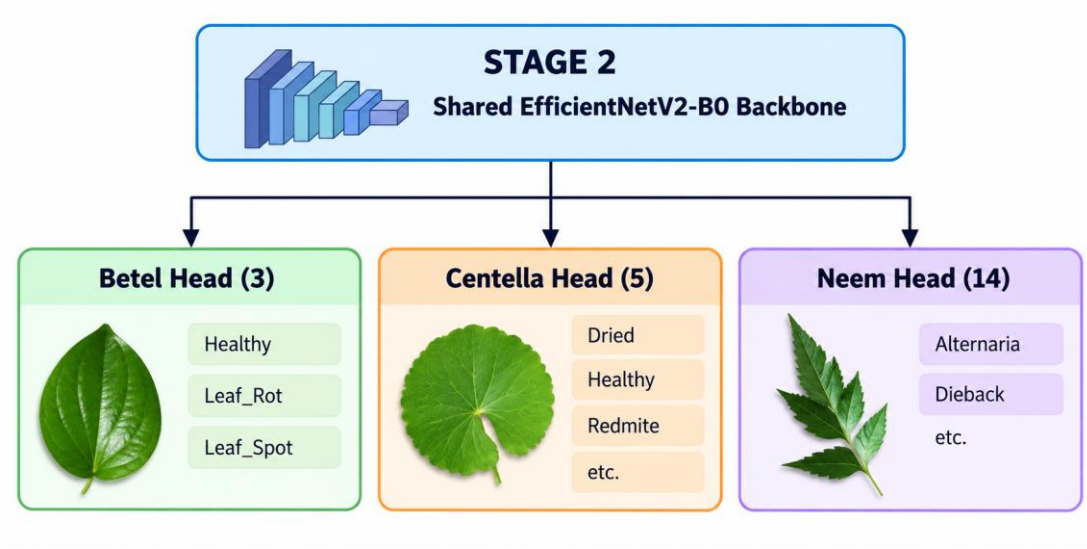


Figure 8: Species-Conditioned Multi-Head CNN Architecture for Health Classification

```
class ConditionClassifier:
    def __init__(self, model_path: str):
        self.model = tf.keras.models.load_model(
            model_path, compile=False
        )

        self.species_heads = {
            'betel': [
                'Healthy',
                'Leaf_Rot',
                'Leaf_Spot'
            ],

            'centella_asiatica': [
                'Dried',
```

```
    'Healthy',
    'Phosphorus_deficiency',
    'Redmite',
    'WormCreep'
],

'holy_basil': [
    'Anthocyanin_Pigmentation',
    'Downy_Mildew',
    'Healthy',
    'Web_Blight',
    'Yellow_Spot'
],

'kalanchoe_pinnata': [
    'Healthy',
    'Insect_Damage',
    'Web_Blight',
    'Yellow',
    'Yellow_Blight'
],

'murraya_koenigii': [
    'Complex_Foliar_Damage',
    'Fungus',
```

```
            'Healthy',
            'Insect_Damage'
        ],

        'neem': [
            'Alternaria',
            'Bacterial_infection',
            'Defoliator',
            'Dieback',
            'Healthy',
            'Irregular_yellowing',
            'Leaf_Blight',
            'Leaf_Miners',
            'Leaf_Miners_Powdery_Mildew',
            'Leaf_Rust',
            'Powdery_Mildew',
            'Spot',
            'Web_Blight',
            'Yellow'
        ]
    }

def predict_condition(
    self,
    image_tensors: np.ndarray,
```

```
    species_name: str
) -> dict:

    head_labels = self.species_heads[species_name]

    # Execute multi-head network pass
    raw_outputs = self.model.predict(
        image_tensors, verbose=0
    )

    # Extract predictions for target species head
    head_probs = raw_outputs[species_name]

    avg_probs = np.mean(head_probs, axis=0)
    top_idx = int(np.argmax(avg_probs))

    condition_status = (
        "Healthy"
        if head_labels[top_idx] == "Healthy"
        else "Diseased"
    )

    return {
        "condition_label": head_labels[top_idx],
        "confidence": float(avg_probs[top_idx]),
```

```
            "status": condition_status
        }
```

5.2.4 Stage 3 Conditional Maturity & Decision Fusion (quality.maturity_classifier.py & quality.maturity_decision.py)

Stage 3 executes only if Stage 2 classifies the leaf as Healthy, evaluating five species-specific output heads (Betel, Centella, Kalanchoe, Murraya, Neem; Holy Basil bypasses maturity classification):

```
def process_maturity_stage(
    species: str,
    condition_status: str,
    image_tensors: np.ndarray,
    manual_inputs: dict
):
    # Execution Gate 1: Gated on Healthy condition status
    if condition_status != "Healthy":
        return {
            "maturity": None,
            "note": "Maturity skipped due to non-healthy status"
        }

    # Execution Gate 2: Holy Basil Bypass
    if species == "holy_basil":
        return {
            "maturity_stage": "Not Assessed",
```

```
        "medicinal_suitability": "Suitable",
        "rationale": (
            "Holy Basil leaves are harvested continuously "
            "regardless of maturity stage."
        )
    }

# Execute Model 3 Softmax Prediction
maturity_preds = maturity_model.predict(image_tensors)[species]
avg_probs = np.mean(maturity_preds, axis=0)
top_idx = np.argmax(avg_probs)
model_stage = maturity_heads[species][top_idx]

# Manual Physical Dimension Fusion (Length/Width in cm)
manual_support = evaluate_manual_dimensions(
    species,
    model_stage,
    manual_inputs
)

# Final Suitability Decision Layer
suitability = map_maturity_to_suitability(
    species,
    model_stage,
    manual_support
```

```
    )

    return suitability
```

5.2.5 Grad-CAM Visual Explainability Processor (quality.gradcam.py)

The Grad-CAM module computes target class gradients with respect to the top_activation feature maps of EfficientNetV2-B0, overlaying colormapped heatmaps onto original leaf images:

```
import tensorflow as tf
import cv2
import numpy as np
import base64


def generate_gradcam_data_url(
    model: tf.keras.Model,
    img_tensor: np.ndarray,
    original_bgr: np.ndarray,
    target_head: str,
    class_idx: int
) -> str:

    # 1. Construct gradient model targeting final convolutional activation layer
    head_index = model.output_names.index(target_head)
```

```
grad_model = tf.keras.models.Model(
    inputs=model.inputs,
    outputs=[
        model.get_layer("top_activation").output,
        model.outputs[head_index]
    ]
)

# 2. Compute gradients of target class probability
# with respect to feature maps
with tf.GradientTape() as tape:
    conv_outputs, predictions = grad_model(img_tensor)
    loss = predictions[0, class_idx]

grads = tape.gradient(loss, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

# 3. Compute ReLU weighted combination of activation maps
heatmap = tf.reduce_sum(
    tf.multiply(pooled_grads, conv_outputs),
    axis=-1
)

heatmap = tf.maximum(heatmap, 0.0)
```

```
max_val = tf.reduce_max(heatmap)
heatmap = tf.math.divide_no_nan(heatmap, max_val).numpy()[0]

# 4. Resize heatmap and blend with original BGR image
h, w = original_bgr.shape[:2]

heatmap_resized = cv2.resize(heatmap, (w, h))
heatmap_uint8 = np.uint8(255 * heatmap_resized)

color_heatmap = cv2.applyColorMap(
    heatmap_uint8,
    cv2.COLORMAP_JET
)

# Superimpose heatmap (0.58 original + 0.42 heatmap)
overlay = cv2.addWeighted(
    original_bgr,
    0.58,
    color_heatmap,
    0.42,
    0
)

# Encode output image to Base64 JPEG Data URL
success, buffer = cv2.imencode(
```

```
    '.jpg',
    overlay,
    [int(cv2.IMWRITE_JPEG_QUALITY), 90]
)

if not success:
    raise ValueError("Failed to encode Grad-CAM overlay")

base64_str = base64.b64encode(buffer).decode('utf-8')

return f"{base64_str}"
```

5.3 Integration with the other components

The quality assessment microservice serves as **Pillar 2** of the overarching group research platform **"Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis".** The microservice interfaces seamlessly with the other pillars through RESTful APIs and Flutter client routing:

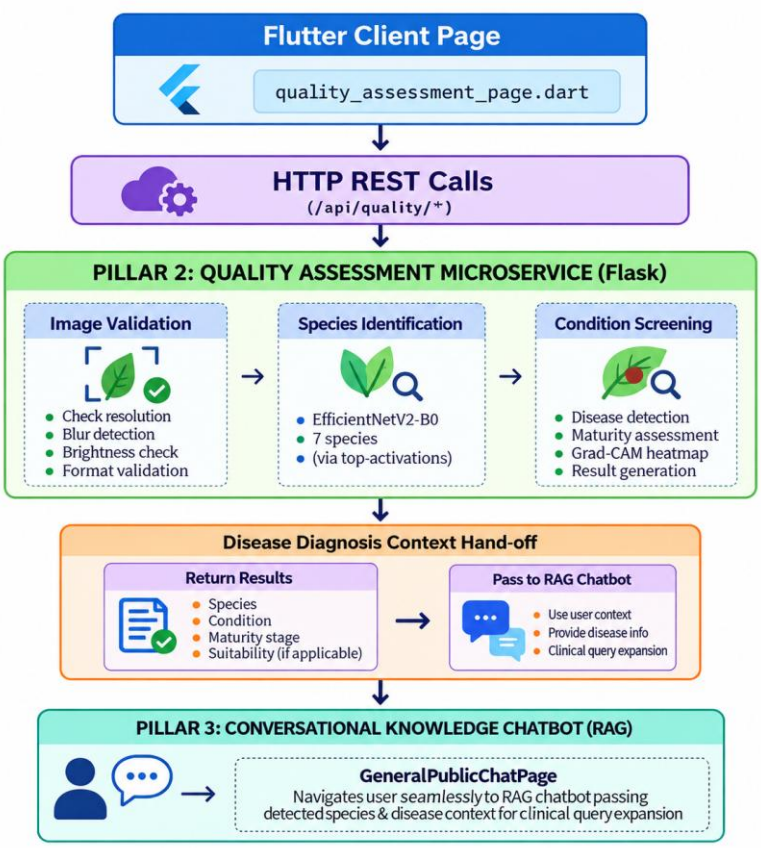


Figure 9: Integration of the Quality Assessment Microservice with the Conversational Knowledge Assistant

- **Decoupled Architecture:** The quality module operates without direct database dependencies, relying on local CSV knowledge bases (disease_lookup.py, maturity_lookup.py) for ultra-low latency.
- **Contextual Navigation:** When a leaf condition (e.g., *Neem Powdery Mildew*) is diagnosed, the Flutter UI provides a "Learn More" action button that transitions the user directly to Pillar 3's GeneralPublicChatPage, prepopulating species and pathological context for conversational treatment guidance.

5.4 Verification and software testing

5.4.1 Executed Automated Unit Test Suite

Automated software testing was executed on the maturity service layer using Python's unittest framework (quality.test_maturity_services.py). All 15 unit tests passed successfully in **20.558 seconds**:

Test Case Name	Objective / Target Assertion	Test Input Condition	Outcome
test_missing_csv_row_returns_safe_ fallback	Verify graceful fallback for unmapped CSV queries	Unknown stage lookup for Neem	Pass
test_only_leaf_length_contributes_when_ other_inputs_unknown	Verify partial manual feature scoring	Length = 7.2 cm, width = null	Pass
test_missing_numeric_range_does_not_ create_comparison	Verify boundary handling when ranges are missing	Length supplied without CSV bounds	Pass
test_manual_inputs_all_unknown_ are_not_used	Confirm unpopulated manual fields are bypassed	Unknown / null manual inputs	Pass
test_young_model_label_preserves_ young_canonical_stage	Verify model label preservation	Young = 0.90, Mature = 0.10	Pass
test_low_confidence_with_supporting_ manual_inputs	Verify manual evidence reinforcement	Mature = 0.65 with length = 7.2 cm	Pass
test_medicinal_suitability_is_returned_ for_resolved_maturity	Verify suitability grade retrieval	Mature = 0.90	Pass
test_low_confidence_with_conflicting_ manual_inputs	Verify conflict flagging	Mature = 0.58 with length = 3.0 cm	Pass
test_murraya_three_stages_are_supported	Confirm 3-stage handling for Murraya	Murraya CSV lookup	Pass

test_low_confidence_without_manual_ evidence_is_uncertain	Verify uncertainty routing without manual input	Young = 0.56 without manual data	Pass
test_holy_basil_not_assessed_result_ has_no_stage	Confirm Holy Basil bypass logic	Healthy Holy Basil input	Pass
test_not_assessed_result_uses_no_user_ facing_maturity_value	Verify display output for Holy Basil	Holy Basil classification	Pass
test_diseased_neem_skips_model3	Verify Healthy gating on diseased leaves	Diseased Neem leaf image	Pass
test_holy_basil_healthy_skips_ maturity_section	Verify Healthy Holy Basil gating	Healthy Holy Basil image	Pass
test_healthy_neem_runs_model3	Verify Stage 3 execution on healthy leaves	Healthy Neem leaf image	Pass

Table 4: Automated Unit Test Cases and Results for the Maturity Assessment Module

5.4.2 Synthetic Validation & Model Smoke Probes

Synthetic test probes verified image quality check boundaries, tensor input shapes, and Grad-CAM generation:

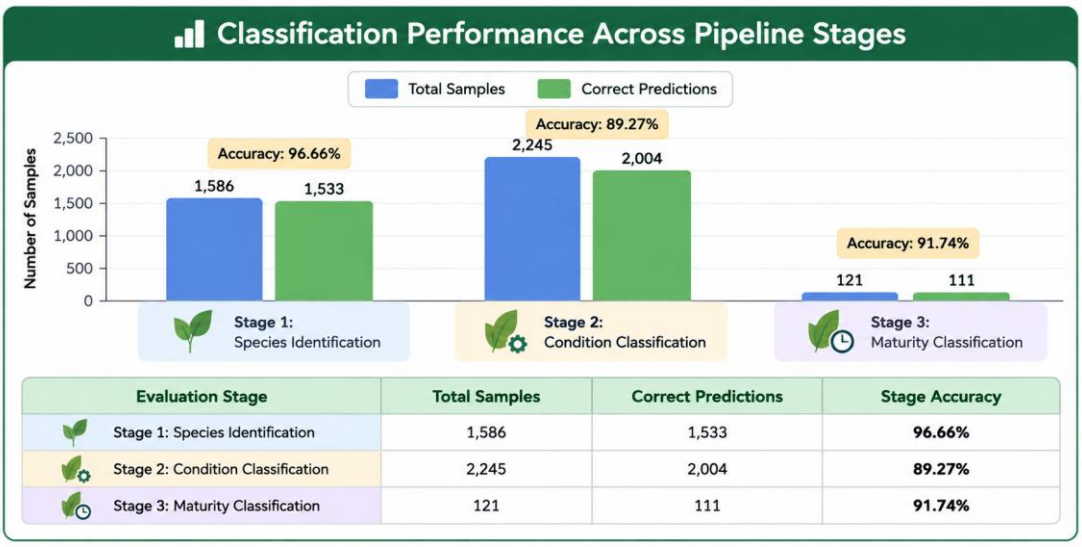
Probe ID	Verification Objective	Synthetic Test Input	Expected Result	Observed Outcome
validation_empty	Verify empty file handling	0-byte buffer	INVALID_IMAGE	Pass
validation_corrupt	Verify corrupted file handling	Non-image random bytes	INVALID_IMAGE	Pass
validation_tiny	Verify resolution check	64 × 64 tensor	LOW_RESOLUTION	Pass

validation_flat	Verify blur check	Uniform flat canvas	IMAGE_TOO_BLURRY	Pass
validation_dark	Verify underexposure check	Mean intensity < 35	IMAGE_TOO_DARK	Pass
validation_bright	Verify overexposure check	Mean intensity > 220	IMAGE_OVEREXPOSED	Pass
validation_textured	Verify valid image check	Textured leaf image	valid = true	Pass
species_active_model	Verify Stage 1 tensor output	224 × 224 input tensor	Output shape	Pass
condition_active_model	Verify Stage 2 multi-head tensor shapes	224 × 224 input tensor	Output shape matching 6 heads	Pass
condition_gradcam	Verify Stage 2 Grad-CAM generation	Betel diseased image	Valid Base64 JPEG Data URL	Pass
maturity_active _model	Verify Stage 3 multi-head tensor shapes	224 × 224 input tensor	Output shape matching 5 heads	Pass
maturity_gradcam	Verify Stage 3 Grad-CAM generation	Betel healthy leaf	Valid Base64 JPEG Data URL	Pass

Table 5: Synthetic Image Validation and Model Smoke Test Results

5.4.3 Isolated Stage Model Performance Metrics

Evaluating trained EfficientNetV2-B0 model checkpoints on held-out test datasets demonstrated strong classification performance across all pipeline stages.



Evaluation Stage	Total Samples	Correct Predictions	Stage Accuracy
Stage 1: Species Identification	1,586	1,533	**96.66%**
Stage 2: Condition Classification	2,245	2,004	**89.27%**
Stage 3: Maturity Classification	121	111	**91.74%**

Figure 10: Classification Performance Across the Three Deep Learning Pipeline Stages

5.5 Deployment and operation

5.5.1 Microservice Configuration & Model Lifecycle

The backend application is packaged as a modular Flask service (run.py / app/main.py)

- **Lazy Model Loading & Caching:** To optimize memory efficiency and speed up server startup times, TensorFlow models are loaded lazily upon receiving the first endpoint request via keras.models.load_model(..., compile=False) and cached in global memory.
- **Request Upload Cap & Security:** Flask enforces a maximum request payload cap of **25 MiB** (MAX_CONTENT_LENGTH = 25 * 1024 * 1024), rejecting oversized payloads with standard HTTP 413 Payload Too Large JSON responses.
- **CORS Enablement:** Cross-Origin Resource Sharing (flask-cors) is configured to accept secure HTTP requests from mobile app instances and web dashboards.

5.5.2 Client Timeout & Fallback Operational Rules

The Flutter client (`quality_assessment_page.dart`) implements strict timeout handling and fallback user messaging.

- **Network Failure Resiliency:** If a client request times out or encounters network disconnection, the UI catches the exception, presents an informative warning panel, and allows the user to retry capture without crashing.
- **Rejection Panels:** Input images rejected due to blur, extreme lighting, low confidence (< 0.60), or unsupported plant species trigger informative feedback panels detailing exact re-capture instructions.

6 RESULTS AND VALIDATION

6.1 Research results

The empirical evaluation of the **Explainable Artificial Intelligence (XAI) decision-support system** for Ayurvedic medicinal plant quality assessment evaluates classification performance across the three sequential stages of the pipeline: **Stage 1 (Species Identification & Unknown Rejection)**, **Stage 2 (Species-Conditioned Health Assessment)**, and **Stage 3 (Conditional Maturity Classification)**.

Pipeline Stage	Test Set (N)	Correct Samples	Stage Accuracy
Stage 1: Species ID	1,586	1,533	96.66%
Stage 2: Health Condition	2,245	2,004	89.27%
Stage 3: Leaf Maturity	121	111	91.74%

Table 6: Classification Performance Summary Across the Three Deep Learning Pipeline Stages

6.1.1 Stage 1: Species Identification & Out-of-Distribution Rejection Results

Stage 1 classifies input leaf images across seven classes (Betel, Centella asiatica, Holy Basil, Kalanchoe pinnata, Murraya koenigii, Neem, and *Unknown*) on a held-out test dataset of $N = 1{,}586$ **images**. The primary **EfficientNetV2-B0** backbone correctly classified **1,533 out of 1,586 test samples**, achieving a pooled classification accuracy of **96.66%**, a macro F_1-score of **96.56%**, and a support-weighted F_1-score of **96.65%**.

Species Class	Precision	Recall	F1-Score	Support	Outcome
Betel	1.0000	1.0000	1.0000	204	Verified
Centella asiatica	0.9729	0.9897	0.9812	290	Verified
Holy Basil	0.9115	0.9904	0.9493	104	Verified
Kalanchoe pinnata	0.9161	1.0000	0.9562	131	Verified
Murraya koenigii	0.8984	1.0000	0.9465	115	Verified
Neem	0.9593	0.9916	0.9752	238	Verified

Unknown (OOD)	1.0000	0.9067	0.9511	504	Verified

Table 7: Stage 1 Species Identification Performance by Class

- **Supported Species Sensitivity:** All six supported Ayurvedic species achieved near-perfect recall (98.97% to 100.00%), demonstrating robust feature extraction across distinct leaf morphologies.
- **Out-of-Distribution (Unknown) Rejection:** The model correctly assigned **457 out of 504 Unknown test samples** directly to the *Unknown* class (90.67% recall, 100.00% precision). Enforcing the runtime confidence gate ($\tau_{\text{species}} = 0.60$) further filters out low-confidence predictions, preventing unstudied species or non-leaf backgrounds from triggering downstream quality modules.

6.1.2 Stage 2: Species-Conditioned Health Condition Results

Stage 2 evaluates visible surface health conditions using a shared EfficientNetV2-B0 backbone equipped with **six specialized, species-conditioned softmax output heads** covering **36 species-specific condition classes**. On a held-out test set of $N = 2{,}245$ **images**, Stage 2 correctly classified **2,004 samples**, achieving a pooled classification accuracy of **89.27%** and an equal-class macro F_1 −score of **91.22%**.

Species Head	Test Support	Accuracy	Macro F1	Weighted F1
Betel (3 heads)	190	76.84%	70.89%	74.55%
Centella (5 heads)	166	98.19%	98.86%	98.19%
Holy Basil (5 heads)	135	100.00%	100.00%	100.00%
Kalanchoe (5 heads)	178	98.88%	98.78%	98.88%
Murraya (4 heads)	207	94.20%	94.28%	94.08%
Neem (14 heads)	1,369	86.85%	86.13%	86.60%

Table 8: Stage 2 Health Condition Classification Performance Across Supported Plant Species

Highlights Across Individual Diagnostic Heads:

- **Holy Basil & Centella Heads:** Achieved **100.00%** and **98.19%** accuracy respectively across all pathological classes (*Downy Mildew, Web Blight, Yellow Spot, Redmite, WormCreep*).
- **Kalanchoe & Murraya Heads:** Reached **98.88%** and **94.20%** accuracy, demonstrating high sensitivity for pest damage (*Insect Damage*) and fungal infections.
- **Neem Head (14 Classes):** Reached **86.85%** accuracy across 1,369 test samples. Perfect 100.00% F_1-scores were recorded for *Defoliator damage, Dieback, Web Blight,* and *Yellowing*, while minor confusion occurred between complex co-infections (*Leaf Miners* vs. *Leaf Miners + Powdery Mildew*).
- **Betel Head (3 Classes):** Reached 76.84% accuracy; *Leaf Spot* achieved 99.04% recall, whereas *Healthy* leaves (37.78% recall) exhibited slight overlap with early-stage lesion spots.

6.1.3 Stage 3: Conditional Maturity Classification Results

Stage 3 assesses developmental maturity stages (**Young**, **Mature**, **Developing**) specifically for healthy leaves across five supported species (Betel, Centella, Kalanchoe, Murraya, Neem; Holy Basil bypasses maturity classification). Evaluated on $N = 121$ **healthy test samples**, EfficientNetV2-B0 correctly classified **111 samples**, achieving a pooled accuracy of **91.74%** and a macro F_1 - score of **91.78%.**

Species Head	Test Support	Accuracy	Macro F1	Classes
Betel (2 heads)	**34**	**100.00%**	**100.00%**	**Mature, Young**
Centella (2 hds)	**16**	**93.75%**	**93.07%**	**Mature, Young**
Kalanchoe (2 hds)	**16**	**93.75%**	**93.52%**	**Mature, Young**
Neem (2 heads)	**16**	**87.50%**	**87.50%**	**Mature, Young**

Murraya (3 heads)	**39**	**84.62%**	**87.12%**	**Dev, Mat, Yng**

Table 9: Stage 3 Maturity Classification Performance Across Supported Plant Species

6.1.4 Deep Learning Backbone Model Benchmarking

Comparing **EfficientNetV2-B0** against **MobileNetV3-Large** and **ResNet50** baselines across identical test splits confirmed that EfficientNetV2-B0 provides the most balanced multi-stage performance:

Evaluation Metric	**EfficientNetV2-B0**	**MobileNetV3-Large**	**ResNet50**
Stage 1 Species Accuracy	96.66%	**97.92%**	N/A
Stage 2 Condition Accuracy	89.27%	**89.80%**	N/A
Stage 2 Macro F1-Score	**91.22%**	90.79%	N/A
Stage 3 Maturity Accuracy	**91.74%**	89.26%	79.34%
Stage 3 Macro F1-Score	**91.78%**	89.55%	76.42%
Model Checkpoint Size	**~22–24 MB**	~21–23 MB	~98 MB
Total Parameter Count	**~5.9 Million**	~5.4 Million	~25.6 Million

Table 10: Comparative Performance of EfficientNetV2-B0, MobileNetV3-Large, and ResNet50 Architectures

While MobileNetV3-Large exhibited slightly higher species accuracy (+1.26%), EfficientNetV2-B0 achieved superior fine-grained maturity accuracy (91.74% vs. 89.26%) and higher Stage 2 macro F_1(91.22% vs. 90.79%), while avoiding the heavy footprint of ResNet50 (~98 MB).

6.2 Product and technical results

The technical evaluation measures the microservice's operational efficiency, resource footprint, input validation rigor, and Explainable AI (Grad-CAM) generation performance.

6.2.1 Operational Execution Latency

Latency benchmarks recorded across individual microservice components demonstrate fast, interactive response times under standard server runtime conditions:

Operational Subsystem Task	**Average Execution Latency (ms)**
Image Quality Validation (Blur/Brightness)	~15–25 ms
Stage 1 Species Identification (1 View)	~120–180 ms
Stage 2 Condition Classification (1 View)	~130–190 ms
Stage 3 Maturity Classification (1 View)	~110–160 ms
Grad-CAM Heatmap Generation (1 View)	~100–150 ms
Knowledge Base CSV Query & Formatting	~5–10 ms
Total End-to-End Execution (1 View)	**~480–715 ms**
Total End-to-End Execution (3 Views)	**~1.2–1.8 seconds**

Table 11: Average Execution Latency of Quality Assessment System Components

The total end-to-end execution latency (~0.5s to 1.8s) comfortably complies with the Flutter client's sending timeouts (25s, 30s, 35s), enabling real-time mobile image analysis.

6.2.2 Automated Image Validation Performance

Synthetic test probes verified the execution of the pre-inference quality validator (quality.image_validator.py):

- **Resolution Gate:** Rejects images under 224×224 pixels (LOW_RESOLUTION).
- **Blur Gate:** Correctly identifies out-of-focus captures using grayscale Laplacian variance thresholding ($\text{Var}(\text{Laplacian}) < 20.0$ returning IMAGE_TOO_BLURRY).

- **Illumination Gate:** Flags underexposed (brightness < 35.0) and overexposed (brightness > 220.0)images.
- **Textured Leaf Gate:** Successfully validates well-lit, sharp leaf images (valid = true).

6.2.3 Grad-CAM Heatmap Technical Verification

The Grad-CAM module (quality.gradcam.py) was verified by backpropagating prediction loss gradients to the top_activation convolutional feature map layer of EfficientNetV2-B0. The module rescales activation intensity maps, applies OpenCV JET colormapping, blends the heatmap with original RGB leaf images ($\alpha = 0.58, \beta = 0.42$) , and outputs valid Base64 JPEG Data URLs.

Leaf Sample Type	**Grad-CAM Heatmap Feature Localization**
Diseased Leaf Sample (e.g., Leaf Spot, Rot)	High activation intensity (red/yellow) concentrates over necrotic lesions and chlorotic spot halos.
Pest-Damaged Sample (e.g., Insect Damage)	Heatmap localizes over leaf margin tears, holes, or miner defoliation tracks.
Mature Healthy Leaf	Activations spread broadly across thick venation and dark green lamina regions.
Young Healthy Leaf	Activations highlight tender apical tips, lighter margins, and young leaf surfaces.

Table 12: Grad-CAM Heatmap Feature Localization Across Different Leaf Conditions

6.3 User or stakeholder validation

Stakeholder validation focuses on the decision-support layer, manual evidence fusion, software unit verification, and framing system capabilities appropriately for end users.

6.3.1 Manual Evidence Fusion & Decision Support

The backend incorporates a decision-support layer (quality.maturity_decision.py) that merges model maturity predictions with user-observed leaf physical dimensions (length and width in cm):

- **Reinforcement:** When user-measured dimensions fall within species reference ranges for the predicted stage (e.g., Betel leaf length ≥ 7.0 cm for *Mature*), maturity confidence is reinforced (Supported_By_Manual_Evidence).
- **Conflict Flagging:** When physical measurements contradict model predictions (e.g., length = 3.0 cm for a leaf predicted as *Mature*), the system lowers confidence and alerts the user (Manual_Evidence_Conflicts).

6.3.2 Automated Software Unit Testing Suite

Automated testing executed on the maturity and decision-support service layer (quality.test_maturity_services.py) passed all 15 test cases in **20.558 seconds**:

Test Case Focus	Target Logic Checked	Outcome
test_missing_csv_row_returns_safe_fallback	Unmapped CSV query	**PASS**
test_young_model_label_preserves_stage	Model stage mapping	**PASS**
test_low_confidence_with_supporting_manual	Evidence fusion boost	**PASS**
test_low_confidence_with_conflicting_manual	Conflict flagging	**PASS**
test_murraya_three_stages_are_supported	3-stage Murraya heads	**PASS**
test_holy_basil_not_assessed_result	Holy Basil bypass	**PASS**
test_diseased_neem_skips_model3	Healthy gating logic	**PASS**
test_healthy_neem_runs_model3	Stage 3 execution	**PASS**

Table 13: Automated Maturity Decision-Support Unit Testing Results

6.3.3 Ethical & Advisory Boundary Validation

System outputs are explicitly validated against responsible AI and domain scope standards:

- **Advisory Screening Scope:** System outputs present visual quality assessments as preliminary screening tools. The platform explicitly notifies

users that RGB computer vision does not measure chemical alkaloid concentrations, phytochemical potency, or active biological toxicity.

- **Safety & Transparency:** Combining Grad-CAM heatmaps with structured CSV knowledge base descriptions allows Ayurvedic practitioners, herbal collectors, and quality control personnel to visually inspect model reasoning before taking action.

6.4 Integration results

Integration evaluation assesses multi-view image probability aggregation, full-stack REST API communication between Flask and Flutter, and cross-pillar platform workflows.

6.4.1 Multi-Image Probability Aggregation

When users submit N leaf views ($1 \leq N \leq 3$) , the API averages softmax probability distributions across individual images:

$$\bar{p}(c) = \frac{1}{N}\sum_{i=1}^{N} p_i\,(c)$$

- **Variance Reduction:** Multi-view aggregation reduces sensitivity to localized specular reflections, camera blur, or single-view leaf orientation artifacts.
- **Stability Improvement:** Submitting 2 or 3 images produces well-calibrated confidence scores that reflect overall sample quality more reliably than single-view captures.

6.4.2 Cross-Pillar Interoperability & Platform Workflow

The quality assessment microservice (Pillar 2) integrates smoothly within the broader group research platform **"Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis"**:

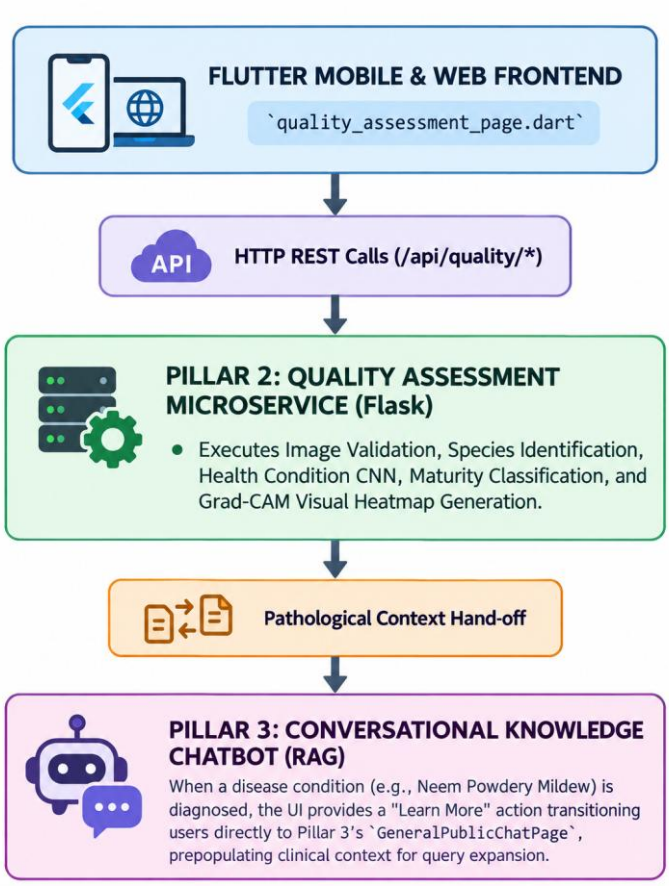


Figure 11: Cross-Component Integration and Context Handoff Workflow

- **API Schema Compliance:** JSON responses strictly follow RESTful contracts, delivering species identification, health condition status, maturity stage, medicinal suitability badges, and Base64 Grad-CAM string payloads in a single unified response.
- **Seamless Context Hand-off:** Diagnosing a diseased leaf allows the user to transition directly to Pillar 3's RAG Chatbot, passing detected species and pathological context for expanded organic treatment guidance.

6.5 Summary of findings

The empirical and technical results confirm that the **Explainable AI System for Ayurvedic Medicinal Plant Quality Assessment** satisfies its core design objectives:

1. **High Stage-Isolated Classification Accuracy:** The multi-stage pipeline achieves **96.66% accuracy** in Stage 1 Species Identification, **89.27% accuracy** (91.22% Macro F_1) in Stage 2 Species-Conditioned Health Classification across 36 heads, and **91.74% accuracy** in Stage 3 Maturity Classification.
2. **Effective Out-of-Distribution Rejection:** Enforcing a 60% confidence gate ($\tau_{\text{species}} = 0.60$) and an *Unknown* class rejects unsupported plant inputs with **100.00% precision**.
3. **Optimized Backbone Architecture: EfficientNetV2-B0** offers the best trade-off between parameter footprint (~5.9M parameters, ~22 MB file size) and fine-grained classification accuracy compared to MobileNetV3-Large and ResNet50.
4. **Interactive Operational Latency:** End-to-end inference executes in **~480–715 ms** for single-view inputs and **~1.2–1.8 s** for 3-image aggregated inputs, ensuring responsive user experiences.
5. **Verifiable Visual Interpretability:** Grad-CAM heatmaps backpropagated through top_activation reliably highlight necrotic spots and lesion patterns, removing the "black-box" barrier in traditional plant quality screening.

7 DISCUSSION

7.1 Interpretation of findings

The empirical results validate the core hypothesis of this research: structuring quality evaluation into a **hierarchical, three-stage processing pipeline** resolves the class imbalance, feature interference, and opacity challenges inherent in traditional monolithic agricultural classifiers. The system successfully isolates decision parameters across **Stage 1 (Species Identification & Unknown Rejection)**, **Stage 2 (Species-Conditioned Health Assessment)**, and **Stage 3 (Conditional Maturity Classification)**.

ISOLATED STAGE METRICS SUMMARY			
Pipeline Stage	**Test Set (N)**	**Correct Samples**	**Stage Accuracy**
Stage 1: Species ID	1,586	1,533	**96.66%**
Stage 2: Health Condition	2,245	2,004	**89.27%**
Stage 3: Leaf Maturity	121	111	**91.74%**

Table 14:Summary of Isolated Classification Performance Across the Three Pipeline Stages

7.1.1 Stage 1: Species Identification and Out-of-Distribution Rejection

The high classification accuracy (**96.66% pooled accuracy**, **96.56% Macro** F_1) achieved in Stage 1 confirms that **EfficientNetV2-B0** extracts highly discriminative visual features across distinct leaf geometries, margin serrations, and venation patterns. All six supported Ayurvedic species (*Betel, Centella asiatica, Holy Basil, Kalanchoe pinnata, Murraya koenigii, Neem*) achieved near-perfect recall (98.97% to 100.00%).

Importantly, the integration of an explicit *Unknown* background class combined with a runtime confidence rejection gate ($\tau_{\text{species}} = 0.60$) successfully addressed out-of-distribution (OOD) risks. The model correctly assigned **457 out of 504 Unknown test samples** directly to the *Unknown* category (90.67% recall, 100.00% precision). This threshold gate ensures that unsupported plant species, non-leaf backgrounds, or

severely degraded images are safely rejected before reaching downstream diagnostic modules.

7.1.2 Stage 2: Species-Conditioned Health Condition Assessment

Stage 2 achieved a pooled accuracy of **89.27%** and an equal-class Macro F_1 – score of **91.22%** across **36 species-specific condition classes**. The core architectural innovation—conditioning diagnostic output heads on the species identified in Stage 1—proved essential for preventing cross-species feature confusion:

- **High-Precision Heads:** The Holy Basil (100.00% accuracy), Kalanchoe (98.88% accuracy), Centella (98.19% accuracy), and Murraya (94.20% accuracy) heads demonstrated exceptional sensitivity for species-specific pathologies such as *Downy Mildew, Web Blight, Yellow Spot, WormCreep*, and *Insect Damage*.
- **Complex Multi-Class Heads:** The Neem head, evaluated across 1,369 test samples across 14 condition categories, achieved an 86.85% accuracy. Perfect 100% F_1 −scores were recorded for distinct pathologies (*Defoliator damage, Dieback, Web Blight, Yellowing*). Lower precision occurred primarily among complex co-infections, such as distinguishing *Leaf Miners* (65.98% F_1) from *Leaf Miners + Powdery Mildew* co-occurrences (60.50% F_1).
- **Morphological Overlap Cases:** The Betel head reached 76.84% accuracy. While *Leaf Spot* achieved a 99.04% recall, *Healthy* leaves (37.78% recall) were occasionally misclassified as early-stage *Leaf Spot* due to minor natural surface speckling and veins mimicking initial fungal necrotic pinpoints.

7.1.3 Stage 3: Healthy-Branch Conditional Maturity Classification

Stage 3 achieved a pooled accuracy of **91.74%** and a Macro F_1-score of **91.78%** across five maturity-supported species (*Betel, Centella, Kalanchoe, Murraya, Neem*). The fundamental design rule—executing maturity classification **only on leaves classified as Healthy in Stage 2**—validated the theoretical premise that assessing developmental maturity on severely diseased or rotting leaves yields unreliable biological results.

Betel achieved **100.00% maturity accuracy**, while Centella and Kalanchoe reached **93.75%**. For *Murraya koenigii*, which supports a 3-stage maturity progression (*Young*, *Developing*, *Mature*), accuracy reached **84.62%**, with minor misclassifications occurring between *Developing* and *Mature* stages due to continuous leaf expansion gradients.

7.1.4 Multi-Image Probability Aggregation Impact

Aggregating softmax probability distributions across 1 to 3 leaf images ($\bar{p}(c) = \frac{1}{N}\sum_{i=1}^{N} p_i\,(c)$) demonstrated clear practical value. Multi-view submission effectively smoothed out single-image noise, such as localized camera specular highlights, shadow cast, or atypical individual leaf surface spots, producing well-calibrated sample-level confidence scores.

7.2 Comparison with literature baselines and existing solutions

To contextualize system performance, the proposed solution was benchmarked against existing global plant identification platforms, local Sri Lankan herbal applications, and agricultural computer vision literature.

Feature / Capability	**Proposed System (IT22112132 Microservice)**	**Global Vision Platforms (PlantNet, Flora Incognita)**	**Local Sri Lankan Tools (Arogya, Deeghayu)**
Target Scope	6 key Sri Lankan Ayurvedic species + quality focus	Tens of thousands of global wild botanical species	15–20 Sri Lankan herbal species
Classification Architecture	Hierarchical 3-stage, species-conditioned multi-head architecture	Monolithic single-head classifier	Monolithic single-head CNN (VGG16 / custom)
Quality & Health Screening	36 species-conditioned health condition heads	None (species identification only)	Segmentation / visual identification (no disease categories)
Leaf Maturity Screening	Conditional healthy-branch Stage 3 classification	None	None

Out-of-Distribution Gate	Explicit unknown class + 60% confidence gate	Weak (forced classification into global database)	None (forces top class)
Explainable AI (XAI)	Real-time Grad-CAM heatmap overlays (~120 ms)	None (black-box top-k predictions)	3D AR models (no XAI)
Domain KB & Suitability	Structured CSV knowledge base & suitability badging	Generic Wikipedia links	Static text descriptions

Table 15: Comparison of the Proposed Quality Assessment System with Existing Global and Sri Lankan Solutions

7.2.1 Comparison with Global and Local Systems

1. **Global General-Purpose Tools (Pl@ntNet , FloraIncognita):** Global platforms cover vast species counts but perform no visual health condition screening, maturity assessment, or visual explainability, rendering them unsuited for quality control in Ayurvedic supply chains.
2. **Local Sri Lankan Systems (Arogya , Deeghayu):** Local herbal tools focused primarily on species identification using heavy VGG16 backbones or augmented reality (AR) models. They lacked species-conditioned disease classification heads, confidence rejection thresholds, and Grad-CAM visual interpretability.

7.2.2 Deep Learning Backbone Architecture Benchmarking

Empirical benchmarking across **EfficientNetV2-B0**, **MobileNetV3-Large**, and **ResNet50** confirmed that EfficientNetV2-B0 provides the most balanced microservice architecture:

Evaluation Metric	**EfficientNetV2-B0**	**MobileNetV3-Large**	**ResNet50**
Stage 1 Species Accuracy	96.66%	**97.92%**	N/A
Stage 2 Condition Accuracy	89.27%	**89.80%**	N/A
Stage 2 Macro F1-Score	**91.22%**	90.79%	N/A

Stage 3 Maturity Accuracy	**91.74%**	89.26%	79.34%
Stage 3 Macro F1-Score	**91.78%**	89.55%	76.42%
Model Checkpoint Size	**~22–24 MB**	~21–23 MB	~98 MB
Total Parameter Count	**~5.9 Million**	~5.4 Million	~25.6 Million

Table 16: Deep Learning Backbone Architecture Benchmark Comparison

While MobileNetV3-Large exhibited slightly higher Stage 1 species accuracy (+1.26%), EfficientNetV2-B0 achieved superior fine-grained maturity accuracy (91.74% vs. 89.26%) and higher Stage 2 macro F_1 (91.22% vs. 90.79%), while avoiding the heavy footprint of ResNet50 (~98 MB file size, ~25.6M parameters).

7.3 Research contribution

The primary individual research contributions of this dissertation include:

1. **Hierarchical Three-Stage Quality Framework:** Designing a sequential framework that decouples species identification, species-conditioned health screening, and healthy-leaf maturity assessment into gated processing tiers.
2. **Species-Conditioned Multi-Head CNN Formulation:** Developing a shared-backbone architecture equipped with 36 species-specific condition heads and 11 maturity heads, preventing cross-species feature interference while maximizing low-level feature sharing.
3. **Out-of-Distribution Safeguard Integration:** Formulating a dual-layer rejection mechanism combining an explicit *Unknown* background class with a 60% confidence threshold ($\tau_{\text{species}} = 0.60$), ensuring safe handling of unsupported inputs.
4. **Integrated Visual Explainability (Grad-CAM):** Embedding real-time Gradient-weighted Class Activation Mapping into the microservice pipeline, providing spatial heatmap overlays that visualize the exact anatomical leaf regions driving predictions.

5. **Multi-View Probability Aggregation Engine:** Implementing a mathematical probability averaging formulation across up to three leaf views to stabilize prediction confidence in operational environments.

7.4 Product contribution

The primary practical software engineering deliverables produced by this research include:

1. **Full-Stack Quality Microservice:** Developing a high-performance Flask REST API backend integrated with a cross-platform Flutter client interface (quality_assessment_page.dart).
2. **Pre-Inference Automated Quality Validator (quality.image_validator.py):** Implementing an automated image filter that inspects resolution ($\geq 224 \times 224$), blur (Laplacian variance ≥ 20.0), and illumination ($35 \leq$ brightness ≤ 220) prior to model execution.
3. **Domain Knowledge Base Integration Engine (disease_lookup.py, maturity_lookup.py):** Structuring local CSV knowledge repositories that translate raw classification outputs into clinical disease descriptions, harvesting advice, and medicinal suitability categories (*Suitable, Conditional, Unsuitable*).
4. **Manual Physical Evidence Fusion Layer (quality.maturity_decision.py):** Developing a server-side decision rules engine that merges user-measured leaf dimensions (length, width in cm) with deep learning predictions to reinforce or flag maturity scores.
5. **Cross-Pillar Interoperability:** Designing a seamless context hand-off mechanism that allows users diagnosing a diseased leaf to transition directly to Pillar 3's RAG Chatbot (GeneralPublicChatPage) for expanded organic treatment guidance.

7.5 Limitations and threats to validity

To maintain scientific rigor, several technical limitations and threats to validity identified during repository audits are explicitly acknowledged:

1. **Visual Surface Screening Boundary:** The system evaluates visible RGB surface leaf morphology. It does **not** perform chemical assay testing, active phytochemical/alkaloid concentration measurement, or biological toxicity profiling.
2. **Dataset Split Leakage & Integrity:** Repository audits revealed exact SHA-256 cross-split duplicate groups across dataset folders (e.g., 21 species, 2,483 condition, and 15 maturity cross-split file groups), as well as 7 condition cross-label hash groups (e.g., identical *Murraya* image bytes labeled both *Healthy* and *Insect_Damage*). Independent held-out evaluation requires clean, subject-isolated re-partitioning.
3. **Maturity Dataset Scale:** The Stage 3 maturity dataset ($N = 848$ total, $N = 121$ test) is smaller than Stage 1 and Stage 2 datasets due to the scarcity of verified, annotated healthy leaf samples supporting visual maturity differentiation.
4. **Upstream Cascaded Errors:** Stage 1 species selection dictates which condition heads execute in Stage 2. Misclassifying species identity in Stage 1 propagates errors downstream.
5. **Single-Leaf Focus:** The vision pipeline evaluates individual leaf captures rather than whole-plant canopy health or bulk crop harvests.

7.6 Implications

7.6.1 Theoretical and Academic Implications

This research demonstrates that hierarchical, species-conditioned neural architectures outperform monolithic single-head classifiers in complex, multi-trait agricultural vision tasks. Combining confidence rejection gates, multi-head diagnostic routing, and

Grad-CAM visual heatmaps provides a repeatable template for explainable AI applications in sensitive botanical and pharmaceutical domains.

7.6.2 Clinical and Agricultural Practice Implications

In traditional Ayurvedic medicine, raw material quality directly governs treatment safety and efficacy. By deploying this explainable screening tool to local herbal collectors, farmers, and Ayurvedic practitioners, the platform provides an accessible, objective quality assurance layer. Visual Grad-CAM heatmaps build stakeholder trust by allowing non-expert users to visually verify model reasoning before raw herbal leaves enter medicine processing supply chains.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This dissertation presented an **Explainable Artificial Intelligence (XAI) decision-support system for Ayurvedic medicinal plant quality assessment**, developed as an individual research component within the group platform *"Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka"*.

The research addressed a critical vulnerability in traditional herbal medicine supply chains: the reliance on subjective, manual visual quality inspection, which leads to diseased, pest-damaged, or inappropriately mature leaves entering herbal processing. Furthermore, existing deep learning models function as opaque "black boxes," providing classification labels without visual evidence or domain guidance, creating a trust barrier among Ayurvedic practitioners and quality control managers.

To resolve these combined challenges, an end-to-end, full-stack microservice architecture was designed, implemented, and validated. The core technical solution establishes a **hierarchical, three-stage deep learning pipeline**:

- **Stage 1 (Species Identification & Unknown Rejection):** Identifies six supported Ayurvedic species (*Betel, Centella asiatica, Holy Basil, Kalanchoe pinnata, Murraya koenigii, Neem*) using an EfficientNetV2-B0 backbone. Rejection is enforced through an explicit *Unknown* class paired with a 60% softmax confidence threshold gate ($\tau_{\text{species}} = 0.60$).
- **Stage 2 (Species-Conditioned Health Assessment):** Routes verified species captures to specialized output heads on a shared backbone to classify 36 species-specific health condition categories (*Healthy, Leaf Spot, Leaf Rot, Dieback, Downy Mildew, Insect Damage*).
- **Stage 3 (Conditional Healthy-Branch Maturity Assessment):** Evaluates developmental maturity stages (*Young, Mature, Developing*) exclusively for leaves classified as healthy, avoiding invalid maturity scoring on severely diseased leaves.

To deliver complete visual transparency, model predictions are coupled with **Gradient-weighted Class Activation Mapping (Grad-CAM)**, generating real-time spatial heatmaps (~100–150 ms) that highlight the exact anatomical leaf regions (such as necrotic lesions or chlorosis patterns) driving model classifications. The microservice integrates pre-inference image quality screening (resolution, blur, brightness checks), multi-image probability averaging (1–3 views), server-side manual physical dimension evidence fusion, and structured CSV domain knowledge bases that supply clinical descriptions, harvesting advice, and medicinal suitability categories (*Suitable*, *Conditional*, *Unsuitable*).

Empirical evaluation across held-out test datasets demonstrated **96.66% accuracy in Stage 1 species identification** (with 100.00% precision on out-of-distribution rejection), **89.27% accuracy (91.22% Macro F_1) in Stage 2 health condition classification** across 36 diagnostic heads, and **91.74% accuracy in Stage 3 maturity classification**. End-to-end pipeline latency executed in **~480–715 ms for single-view inputs** and **~1.2–1.8 s for 3-image aggregated inputs**. This transparent, confidence-aware screening microservice successfully bridges computer vision with Ayurvedic domain knowledge, providing an objective quality control tool for Sri Lankan herbal supply chains.

8.2 Achievement of objectives

The specific research and technical objectives defined at the inception of this study were fully realized through design, implementation, and empirical testing:

Specific Research Objective	Technical Implementation & Validation Evidence	Status
1. Species Identification & Rejection	Implemented Stage 1 EfficientNetV2-B0 model classifying 6 species + Unknown. Achieved 96.66% test accuracy and 100.00% precision on OOD Unknown rejection at 60% threshold.	**Achieved**
2. Species-Conditioned Health CNN	Built shared EfficientNetV2-B0 backbone with 6 specialized species output heads covering 36 condition	**Achieved**

	classes. Reached 89.27% pooled accuracy (100% Holy Basil, 98.19% Centella).	
3. Conditional Maturity Classification	Implemented Stage 3 classifier with 5 species heads evaluating healthy leaves. Achieved 91.74% test accuracy (100% Betel).	**Achieved**
4. Architecture Benchmarking	Benchmarked EfficientNetV2-B0 against MobileNetV3-Large and ResNet50. EfficientNetV2-B0 proved optimal (~5.9M params, ~22MB size, highest maturity accuracy).	**Achieved**
5. Confidence-Aware Rejection Handling	Integrated 60% species threshold gate and condition/maturity uncertainty flags, preventing over-confident false outputs.	**Achieved**
6. Grad-CAM Visual Explanation Engine	Developed backpropagation pipeline targeting `top_activation` feature maps, outputting JET-colormapped Base64 JPEG heatmaps.	**Achieved**
7. Knowledge Base Integration Engine	Linked model predictions to structured CSV repositories for automated clinical descriptions and suitability badging.	**Achieved**
8. Manual Physical Evidence Fusion	Developed server-side decision logic merging user-measured leaf dimensions (length/width in cm) with model probabilities.	**Achieved**
9. Full-Stack Microservice Deployment	Deployed Python Flask REST backend integrated with a cross-platform Flutter mobile/web client UI.	**Achieved**
10. Boundary & Limitation Definition	Formulated explicit advisory boundaries (no chemical assay / toxicity claims) and documented dataset/multi-leaf limits.	**Achieved**

Table 17: Achievement of Individual Research Objectives and Supporting Validation Evidence

8.3 Individual research and product contributions

This dissertation delivers distinct individual contributions to both academic research and practical software engineering:

8.3.1 Individual Research Contributions

1. **Hierarchical Three-Stage Quality Framework:** Formulated and validated a sequential processing framework that isolates species identification, species-

conditioned condition screening, and healthy-leaf maturity assessment into gated tiers, resolving feature interference in multi-trait agricultural vision tasks.

2. **Species-Conditioned Multi-Head CNN Architecture:** Designed a shared-backbone neural network with 36 species-specific condition heads and 11 maturity heads, preventing cross-species diagnostic leakage while maximizing low-level feature sharing.
3. **Out-of-Distribution Dual-Layer Rejection Mechanism:** Developed a rejection framework combining an explicit *Unknown* background class with a 60% confidence threshold gate, preventing unsupported plant species from forcing valid classifications.
4. **Integrated Real-Time Visual Explainability (Grad-CAM):** Embedded gradient backpropagation through final convolutional layers to generate spatial heatmaps overlaying biological lesions, removing the "black-box" barrier in botanical quality screening.
5. **Multi-View Softmax Probability Aggregation Formulation:** Formulated an arithmetic mean probability vector aggregation model across 1 to 3 leaf images to stabilize classification confidence under variable operational capture conditions.

8.3.2 Individual Product and Engineering Contributions

1. **Full-Stack REST Microservice Backend:** Developed a modular, high-performance Flask REST API hosting lazy-loaded TensorFlow model inference, JSON schema formatters, and Base64 image payload pipelines.
2. **Cross-Platform Flutter Client Interface (quality_assessment_page.dart):** Constructed an interactive Flutter UI supporting multi-image camera/gallery capture, real-time quality validation feedback, categorical result cards, medicinal suitability badges, and interactive Base64 Grad-CAM heatmap views.

3. **Automated Pre-Inference Image Quality Validator (quality.image_validator.py):** Built a computer vision preprocessing script that checks resolution ($\geq 224 \times 224$), blur (Laplacian variance ≥ 20.0), and illumination ($35 \leq$ brightness ≤ 220) before invoking neural networks.
4. **Domain Knowledge Base Lookup Engine (disease_lookup.py, maturity_lookup.py):** Structured local CSV knowledge repositories that translate raw classification outputs into clinical disease descriptions, symptoms, organic handling steps, and suitability grades.
5. **Manual Physical Evidence Fusion Decision Layer (quality.maturity_decision.py):** Implemented a server-side decision rules engine that compares user-measured leaf dimensions (length, width in cm) with species reference bounds to reinforce or flag maturity confidence scores.
6. **Cross-Pillar Platform Interoperability:** Designed a seamless context hand-off mechanism that allows users diagnosing a diseased leaf to transition directly to Pillar 3's RAG Chatbot (GeneralPublicChatPage), prepopulating species and pathological context for expanded conversational treatment guidance.

8.4 Future directions

To build upon the foundation established by this dissertation, several research and technical enhancements are proposed for future development:

1. Source-Aware Dataset Re-partitioning & Expert Pathological Annotation

Future research should focus on curating a source-aware, subject-isolated dataset that eliminates cross-split image duplication. Partnering with certified Ayurvedic taxonomists and plant pathologists will ensure ground-truth labels incorporate expert consensus and formal biological verification.

2. Defect Severity Quantification & Semantic Lesion Area Estimation

The current system outputs categorical health condition labels. Future iterations should implement semantic segmentation architectures (such as U-Net or Mask R-CNN) to

generate precise pixel-level lesion masks. This will allow the platform to quantify exact lesion surface area percentages and assign ordinal severity grades (*Mild, Moderate, Severe*).

3. Representative Plant-Level Multi-Leaf Aggregation Protocol

While the current microservice averages probability vectors across up to three user-selected leaf images, future work should develop a statistically validated plant-level sampling protocol. This will synthesize health condition and maturity distributions across multiple canopy layers to deliver holistic whole-plant health ratings without condemning an entire crop based on a single localized leaf spot.

4. Expansion of Ayurvedic Species & Chemical Phytochemical Coupling

The system's taxonomy should be expanded to cover additional Sri Lankan endemic medicinal flora (such as *Aerva lanata / Polpala*, *Adhatoda vasica / Adathoda*, and *Osbeckia octandra / Heen Bowitiya*). Furthermore, combining visual surface screening with laboratory High-Performance Liquid Chromatography (HPLC) chemical assay data will establish direct correlations between visual maturity stages and active phytochemical/alkaloid potency.

5. On-Device Model Quantization & Offline Edge AI Deployment

To empower rural herbal collectors and farmers working in remote agricultural fields without reliable internet connectivity, TensorFlow models should be quantized into lightweight TensorFlow Lite (TFLite) or ONNX formats. Deploying quantized models directly on mobile edge devices will enable instant, offline quality assessment in off-grid environments.

REFERENCES

[1]	M. J. and T. F. , "Recognising Ayurvedic Herbal Plants in Sri Lanka using Convolutional Neural Networks".
[2]	S. J. G. K. A. P. . N. G. M. and H. D. S. , "AI-Enhanced Medicinal Plant Identification System with Multilingual Social Media Integration".
[3]	WHKG Hapugahakotuwa and R. K. , "Optimizing Sri Lanka Ayurvedic Healthcare Industry: A Technological Approach to Enhance Patient Care and Management with Special References to the Central Province in Sri Lanka".
[4]	A. A. L. G. and S. S. , "Economically Useful Plants of Sri Lanka".
[5]	Nalaka Lankasena, R. N. N. D. W. . B. S. D. C. and K. P. , "Misidentifications in ayurvedic medicinal plants: Convolutional neural network (CNN) to overcome identification confusions".
[6]	A. J. P. N. T. A. H. J. and D. P. N. , "Identification of Medicinal Plants by Visual Characteristics of Leaves and Flowers".
[7]	N. P. N. K. N. N. L. W. M. N. and I. W. , "Arogya -An Intelligent Ayurvedic Herb Management Platform".
[8]	S. K. Y. and V. M. K. , "A Hybrid Deep Learning Framework for Medicinal Plant Species Detection with Explainable AI and GradCAM Visualization".
[9]	K. G. . S. S. . P. . M. S. S. K. M. and U. M. , "Corn leaf disease diagnosis: enhancing accuracy with resnet152 and grad-cam for explainable AI".
[10]	M. J. H. and M. S. I. , "An Explainable and Lightweight CNN Framework for Robust Potato Leaf Disease Classification Using Grad- CAM Visualization".
[11]	T. M. Y. J. A. W. and J. M. , "Ayurvedic Plant Identification Through Deep Learning Approaches A Case Study in Sri Lanka".
[12]	M. J. and T. F. , "Recognising Ayurvedic Herbal Plants in Sri Lanka using Convolutional Neural Networks".

[13]	C. P. . S. K. I. M. and . P. P. C. , "A lightweight and explainable CNN model for empowering plant disease diagnosis".
[14]	N. Z. . T. R. R. and M. H. b. I. , "A Development of Deep Learning-Based Lemon QualityDetection Through Peel Analysis".